

Technical Design, System Architecture & Engineering Documentation

An Enterprise-Grade, AI-Powered Home & Family Management Platform.
A comprehensive blueprint of system layers, database configurations, UML schemas, and native background sync operations.

DOCUMENT VERSION

v1.0.0-Enterprise

BUILD NUMBER

Build 1042-Prod

PREPARED BY

Core Engineering & Architecture Group

DATE OF ISSUE

July 7, 2026

CHAPTER 02

Table of Contents

Structural Outline and Index of the Architecture Specifications

Welcome to the **MS Family Technical Design & Architecture Documentation**. Use the links below to navigate directly to each chapter section.

03. Executive Summary	4
04. Project Overview	8
05. Functional Requirements	12
06. Non-Functional Requirements	19
07. System Architecture	25
08. C4 Model Blueprint	31
09. High-Level Layered Architecture	38
10. Low-Level Component Design	44
11. Database Design & ERD	52
12. Database Physical Schema	58
13. Core User Flows	68
14. Application Lifecycle Flow	76
15. Authentication & Lock Sequence	80
16. Expense & OCR Ingestion Sequence	84
17. Bill Scheduler & Alert Sequence	88
18. AI Assistant Reasoning Sequence	92
19. Secure Document Ingestion Sequence	96
20. Vehicle Service & Alert Sequence	100
21. Medication Reminder Sequence	104
22. REST API Specifications	108
23. Security & Threat Architecture	118
24. Frontend Core Architecture	126

25. Backend Core Architecture	131
26. State Management & Persistence	135
27. DevOps & CI/CD Pipeline	139
28. Deployment Architecture Blueprint	143
29. Horizontal & Vertical Scalability	146
30. Performance Tuning & Optimization	150
31. Observability & Monitoring	154
32. Verification & Testing Strategy	158
33. Codebase Directory Tree	162
34. Configuration & Environments	166
35. Design System & Tokens	170
36. UML Core Blueprints	174
37. Architecture Decision Records (ADRs)	182
38. AI & LLM Engineering Pattern	188
39. Product Evolution Roadmap	192
40. Appendix & Bibliography	196

CHAPTER 03

Executive Summary

Vision, Mission, and Platform Value Proposition

Summary: This chapter outlines the strategic objectives, problem space, and the business case for the MS Family platform.

Project Identity

- **System Title:** MS Family Home & Family Management Platform
- **Document Version:** v1.0.0-Enterprise
- **Authoring Body:** Core Software Engineering & Architecture Group

1. System Vision

The MS Family platform addresses the fragmentation of household logistics and family financial operations. By combining modern web architectures, native mobile integrations, and generative AI interfaces, the platform provides a unified workspace for managing shared household resources.

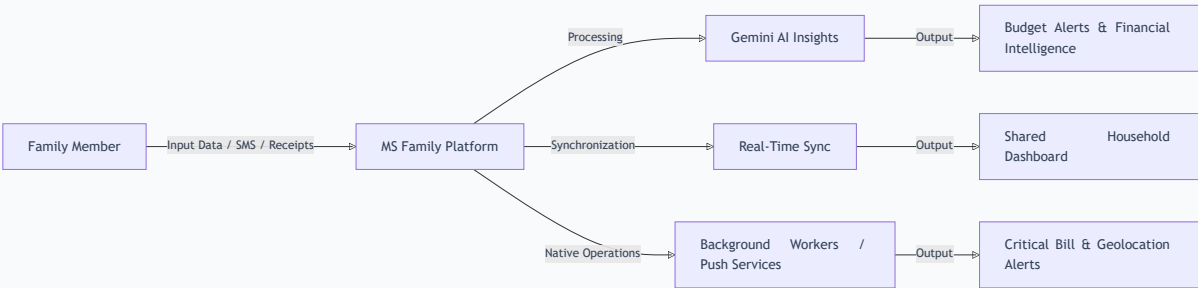


Diagram 1: High-Level Platform Data and Feedback Loop

2. Mission

The platform coordinates household management by: 1. Fusing shared ledgers with automatic native SMS background receipt detection. 2. Indexing documents in a secure vault with Gemini-based metadata

extraction. 3. Automating medication and loan EMI reminders via background scheduling services. 4. Providing real-time family location tracking and peer-to-peer WebRTC calls.



Strategic Alignment

MS Family consolidates multiple household tools (finance ledgers, document storage, calendars, and location sharing) into a single, secure application.

CHAPTER 04

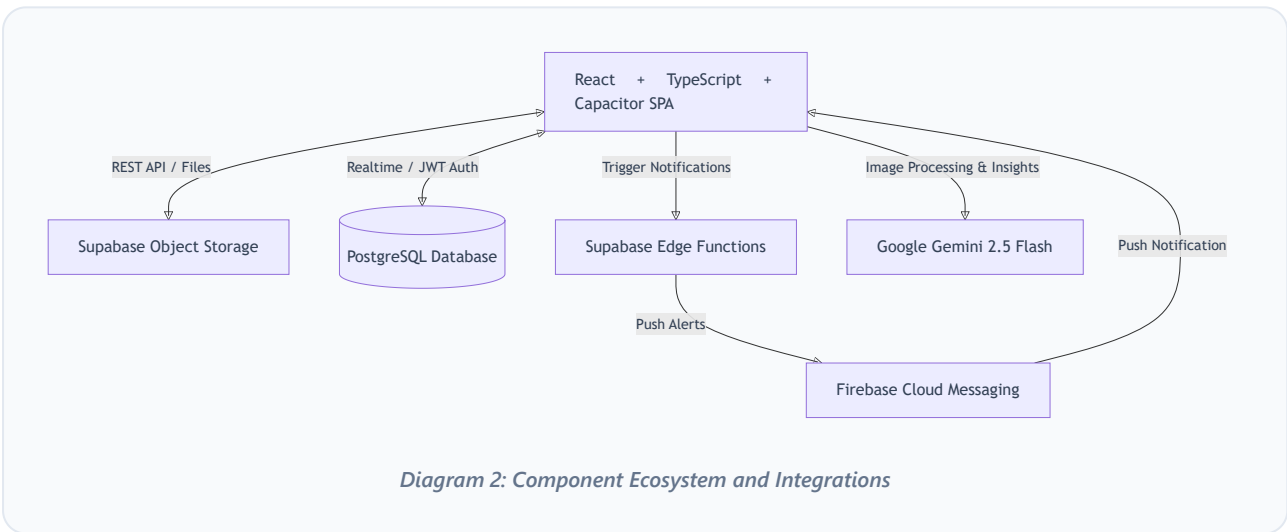
Project Overview

Context, Ecosystem Structure, and Module Definitions

Summary: An introduction to the software modules, dependencies, and deployment structure of the MS Family system.

1. System Context

The MS Family platform is built as a responsive Single-Page Application (SPA) using React, Vite, TypeScript, and Tailwind CSS. It is wrapped as a native hybrid app via Capacitor, allowing access to native Android APIs.



2. Module Specifications

Financial Ledger & Analytics

Tracks income, expenses, and savings goals linked to user profiles. Features Recharts-based visual graphs and monthly resets.

Smart SMS Ingestion

Native Android receivers (`SmsBackgroundReceiver`) capture banking transaction SMS alerts, parse details locally, and upsert records to Supabase.

Secure Document Vault (MyProofs)

Uploads receipt and prescription files to Supabase Storage, using the Gemini API to run OCR and extract metadata (merchant, total, and document summary).

Location Tracking & Geolocation

Tracks family member locations in real-time, displaying them on a shared Leaflet map with battery status reports.

WebRTC VoIP calling

Provides direct peer-to-peer voice calls using Supabase Realtime channels for signaling.



Architecture Target

Offline availability is supported by caching data in a local Room SQLite database before syncing changes with Supabase.

CHAPTER 05

Functional Requirements

Feature Requirements, Priority Matrix, and Acceptance Criteria

Summary: Details the system's functional requirements, feature specifications, and acceptance criteria.

1. MoSCoW Prioritization Matrix

Table 1: Core Functional Priority Matrix

REQUIREMENT ID	MODULE	FEATURE DESCRIPTION	PRIORITY	DEPENDENCIES
FR-FIN-01	Finance	Manual transaction entry (income, expense, savings)	Must Have	Auth-01
FR-FIN-02	Finance	Monthly ledger resets and category aggregation	Must Have	FR-FIN-01
FR-FIN-03	Finance	Category breakdown graphs and historic search	Must Have	FR-FIN-01
FR-SMS-01	SMS Sync	Background SMS intercepting and parsing engine	Must Have	Device Perms
FR-SMS-02	SMS Sync	Verification queue for low-confidence parses	Should Have	FR-SMS-01
FR-DOC-01	Vault	Document upload with title, category, and pin	Must Have	Storage-01
FR-DOC-02	Vault	Gemini-powered OCR metadata extraction and summary	Should Have	FR-DOC-01, Gemini-API
FR-GPS-01	Tracking	Live location sharing with battery status on map	Must Have	Device GPS
FR-CALL-01	Calling	P2P VoIP audio calls between family members	Should Have	WebRTC, Realtime
FR-LN-01	Loans	EMI scheduling and payment status tracking	Must Have	FR-FIN-01
FR-LN-02	Loans	Automated notifications for upcoming EMIs	Must Have	Local Notifs
FR-SAV-01	Savings	Track physical assets (Gold/Silver)	Should Have	FR-FIN-01
FR-SAV-02	Savings	Fetch daily gold rates and trigger price alerts	Could Have	Gemini-API

2. Detailed Acceptance Criteria & Business Rules

FR-SMS-01: Native SMS Parsing Engine

- **Business Rule:** Only SMS messages from registered financial institutions may be processed. Personal messages must be ignored.
- **Acceptance Criteria:**
- The native `SmsBackgroundReceiver` intercepts incoming messages.
- The text is parsed using regex patterns in `SmsParserEngine`.
- If the confidence score is $\geq 70\%$, the transaction is auto-inserted and an in-app `SmsToast` is shown.
- If the confidence is $< 70\%$, the record is routed to the `PendingSmsTransactions` confirmation queue.



Security & Consent

SMS parsing runs entirely on-device. Personal messages are ignored by the parser, preserving user privacy.

CHAPTER 06

Non-Functional Requirements

Performance Targets, Reliability, Security, and Compliance

Summary: Outlines the non-functional requirements, latency thresholds, security standards, and device support targets.

1. Performance & Latency Targets

- **Initial Page Load:** The app loader must resolve in < 1.5 seconds on 3G networks.
- **Cached Database Queries:** Local Room SQLite queries must display data within 300 ms .
- **API Response Time:** Supabase queries and Edge Functions must respond within 250 ms .
- **Gemini OCR Processing:** Image uploads and OCR analysis must complete within 5.0 seconds.

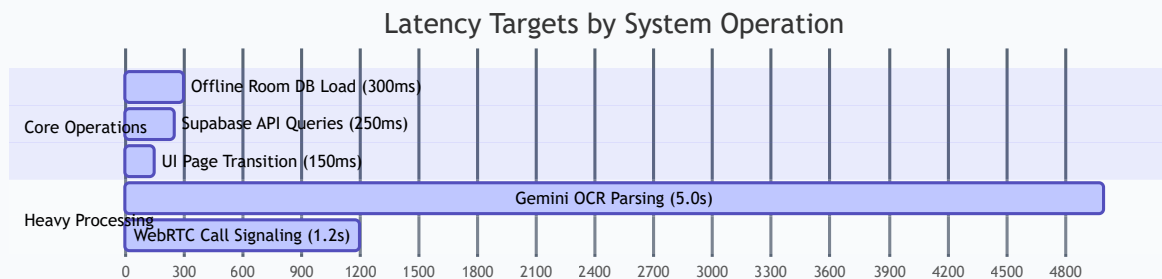


Diagram 3: System Latency Target Budgets

2. Availability & Reliability

- **System Uptime:** Hosted services must maintain $\geq 99.9\%$ availability.
- **Offline Fallback:** When offline, the app must route write operations to the local cache (`TransactionCachePlugin`) and sync them once the connection is restored.
- **Network Monitoring:** Active network state is tracked via `@capacitor/network` to handle offline-to-online transitions.

**Sync Retries**

The sync worker uses an exponential backoff strategy for failed sync operations to prevent overloading the database.

CHAPTER 07

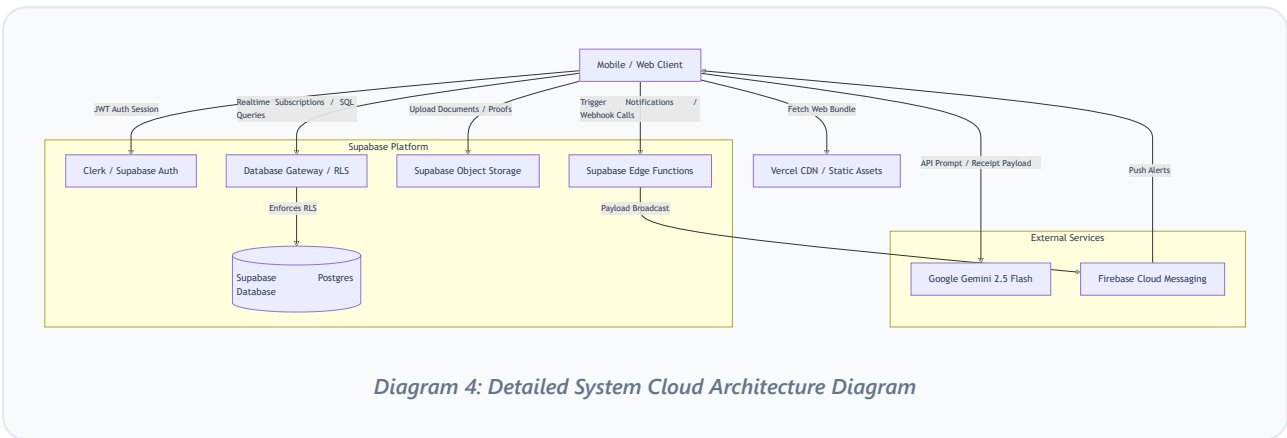
System Architecture

Cloud Topology, Client-Server Paths, and Processing Nodes

Summary: Outlines the cloud architecture, API gateways, security policies, and integrations of the MS Family system.

1. Cloud Architecture Blueprint

The platform uses a decoupled serverless architecture, separating database operations, background tasks, and AI integrations.



2. Core Processing Nodes

Client Edge Layer

Vite bundles assets for web and mobile. On Android, Capacitor bridges JavaScript requests to Kotlin workers (such as GPS tracking, Biometrics, and SMS receivers).

API & Security Gateway (Supabase)

Intercepts client requests, validates JWT tokens, and evaluates Row-Level Security (RLS) policies against the target rows before executing database queries.

Serverless Layer

Deno Edge Functions handle background notifications via FCM and fetch external currency/gold rates.



ADR Reference: Decoupled Gateways

Using Supabase's direct PostgREST API reduces network hops, routing read/write actions directly to PostgreSQL through RLS policies. Detailed decisions are documented in [ADR-002](#).

CHAPTER 08

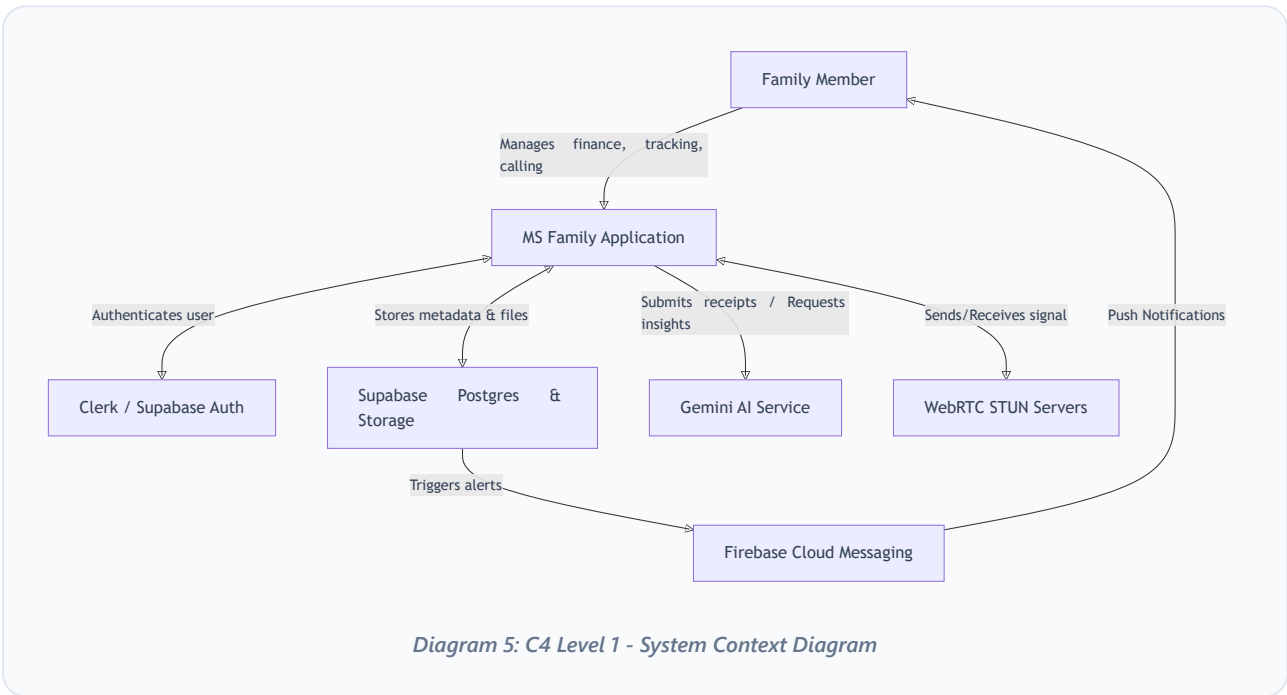
C4 Model Blueprint

C4 Context, Containers, Components, and Code Views

Summary: Displays C4 blueprints illustrating system contexts, containers, components, and code interfaces.

1. Level 1: System Context Diagram

Shows how the system interacts with users and external services.



2. Level 2: Container Diagram

Explains the logical containers that compose the MS Family system.

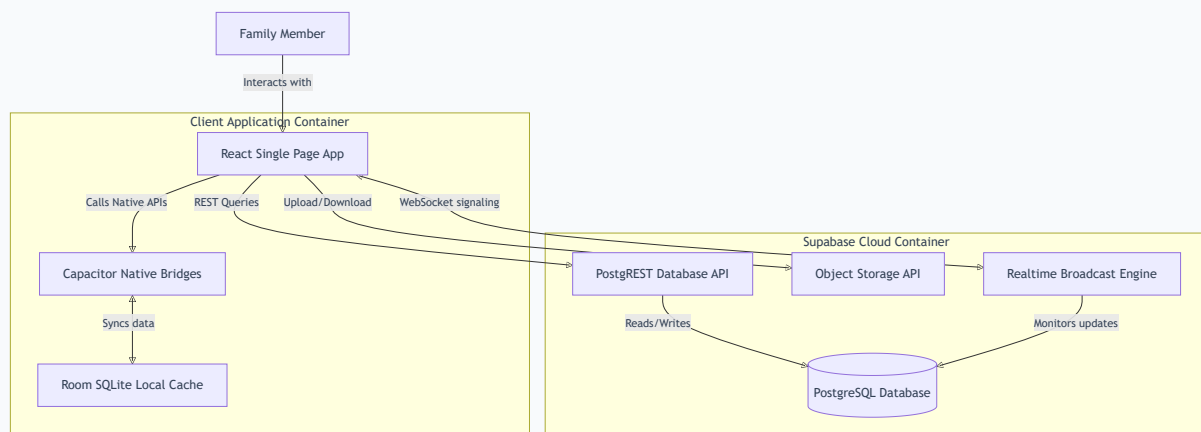


Diagram 6: C4 Level 2 - Container Diagram

3. Level 3: Component Diagram (Frontend & Client)

Illustrates component relationships inside the client application wrapper.

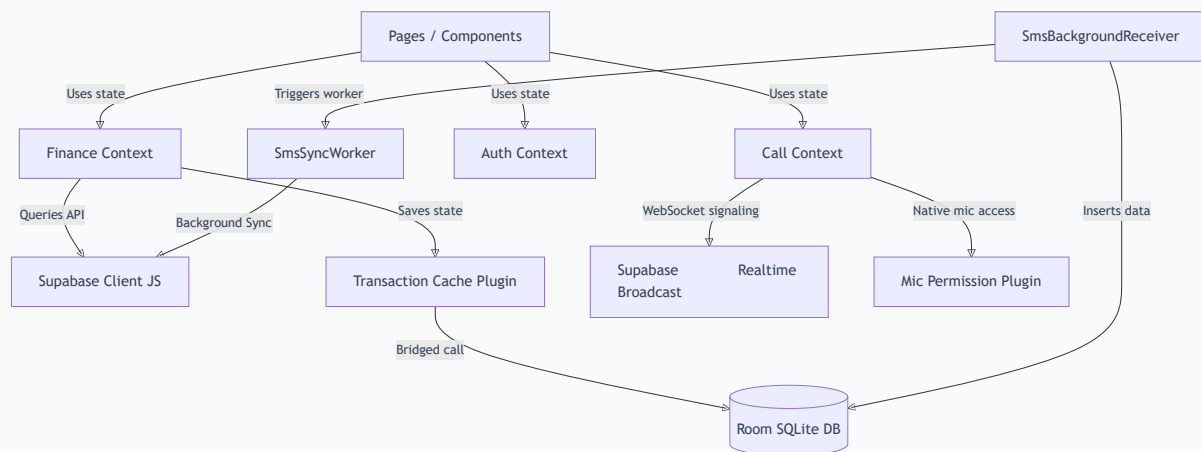


Diagram 7: C4 Level 3 - Component Diagram

CHAPTER 09

High-Level Layered Architecture

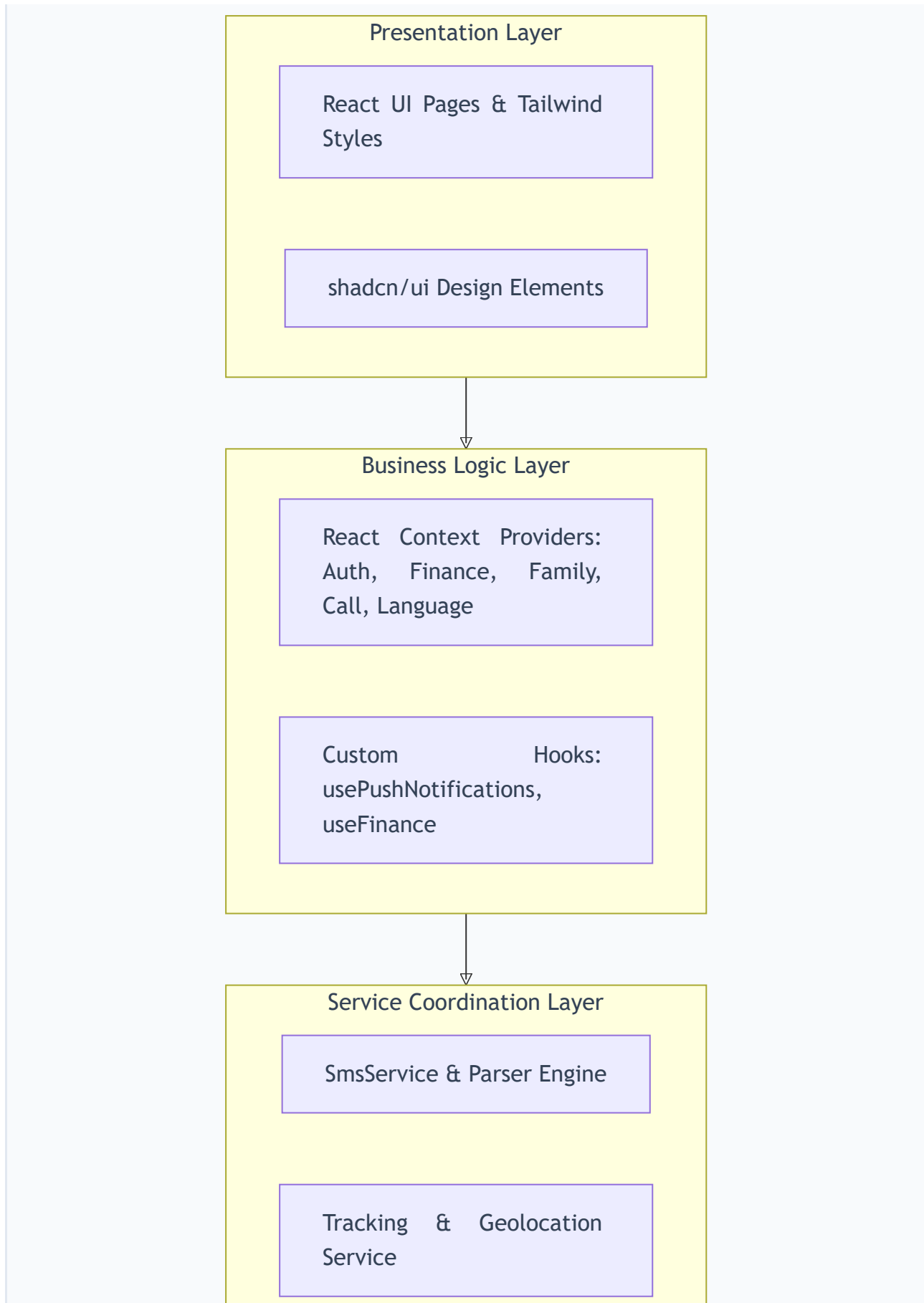
Presentation, Business, Service, Cache, and Database Layering Patterns

Summary: Outlines the n-tier architecture pattern of the MS Family system, showing the boundary responsibilities for each layer.

1. N-Tier Layering Model

The platform separates UI components from core business logic, background workers, and database operations.





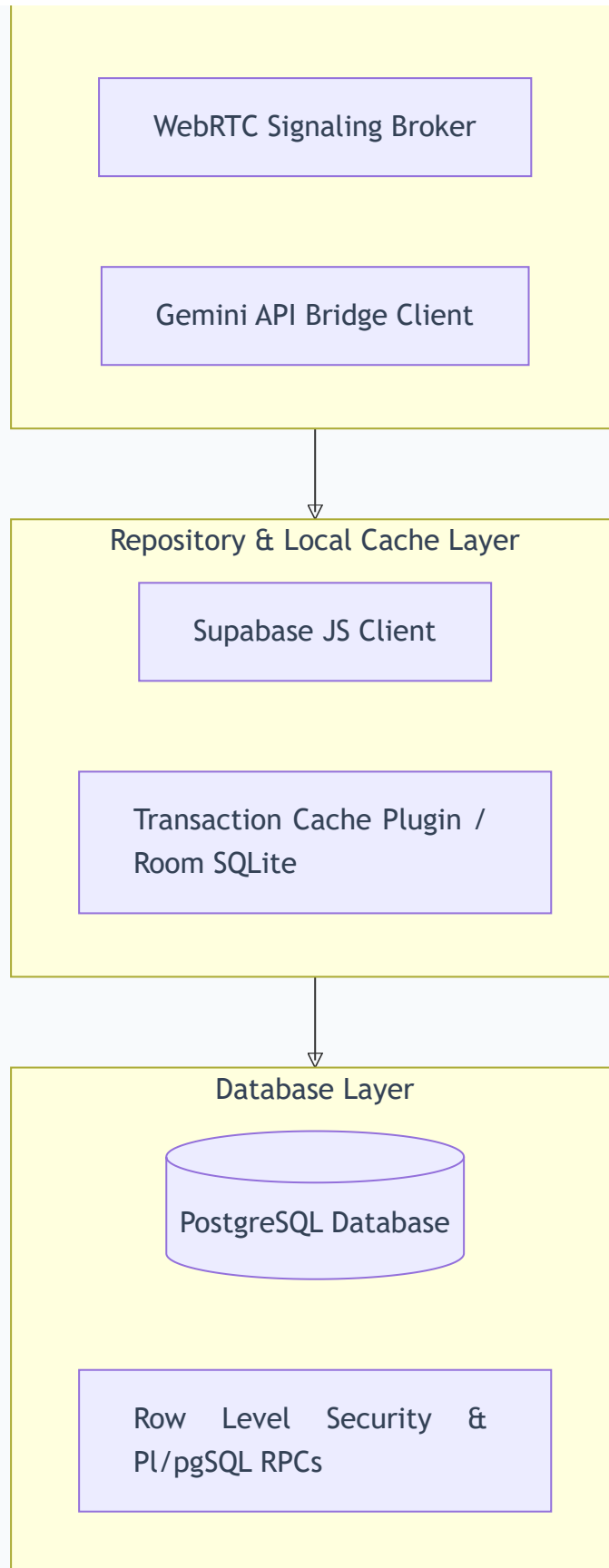
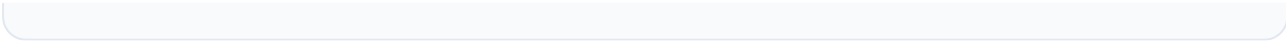


Diagram 8: MS Family Layered Architecture Blueprint



2. Layer Definitions

Presentation Layer

Builds the UI using React, Tailwind CSS, and shadcn/ui. Handles user input, state visualization, page routing, and charts.

Business Logic Layer

Encapsulates transaction business rules, balance calculations, WebRTC signaling state transitions, and user session context.

Service Coordination Layer

Bridges JavaScript frameworks with native phone APIs (such as geolocation background services and SMS interceptors).

Repository & Cache Layer

Coordinates data reads and writes, routing queries to local Room databases when offline.

CHAPTER 10

Low-Level Component Design

Bridge interfaces, Room structures, and background Kotlin workers

Summary: Details the native plugins, Room database interfaces, and background Kotlin/Java worker scripts.

1. Code Execution Map

Illustrates the interaction between JavaScript contexts and native Kotlin engines.

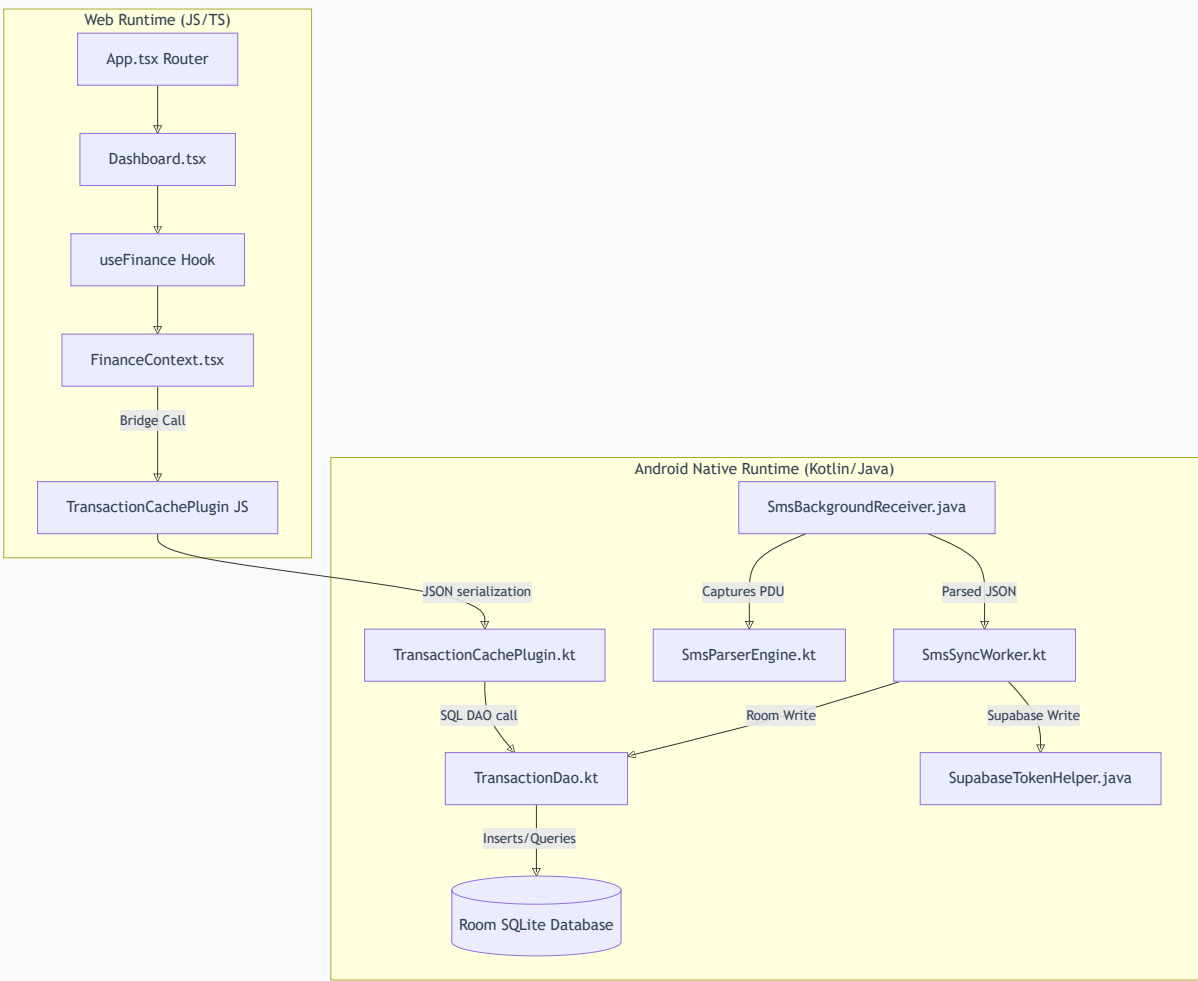


Diagram 9: Code Components Interaction Diagram

2. Native Component Specifications

SmsParserEngine.kt

Runs regular expression parser trees to extract transaction types, amounts, and merchant details from SMS messages.

android/app/.../SmsParserEngine.kt

KOTLIN

```
// Pre-compiled regex models for standard banking alert patterns
val amountRegex = Regex("(?i) (? :rs\\.?|inr|amt) \\s* ( [\\d,]+\\.?\\d* )")
val debitRegex = Regex("(?i) (? :debited|spent|paid|withdrawn|charged)")
val creditRegex = Regex("(?i) (? :credited|deposited|received|added)")
```

TransactionCachePlugin.kt

Exposes SQLite methods to the React layer, translating JavaScript objects into database rows.

CHAPTER 11

Database Design & ERD

Relational Structure, Keys, Constraints, and Indexing Strategies

Summary: Outlines the relational schema database model, indexes, and normalization configurations.

1. Database Entity Relationship Diagram

Displays the schema tables, primary/foreign keys, and data relationships.



Diagram 10: Physical Database Entity Relationship Diagram (ERD)

2. Integrity & Performance Design

Normalization (3NF)

The schema uses 3NF to avoid data duplication. Junction tables (such as `family_members`) decouple user profiles from family structures.

Performance Indexing

B-tree indexes are configured for columns that are queried frequently: `* idx_transactions_member_id on transactions(member_id) * idx_family_members_user_id on family_members(user_id) * idx_my_proofs_user_id on my_proofs(user_id)`

CHAPTER 12

Database Physical Schema

PostgreSQL Table Structures, Data Types, Constraints, and Columns

Summary: Technical details for all tables, fields, nullability, constraints, and column descriptions.

1. Table: `profiles`

Stores user profile information.

Table 2: Profiles Database Schema

FIELD	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
<code>id</code>	<code>UUID</code>	No	<code>PRIMARY KEY</code>	Links to Supabase <code>auth.users(id)</code> .
<code>name</code>	<code>TEXT</code>	Yes	-	Display name of the user.
<code>username</code>	<code>TEXT</code>	Yes	<code>UNIQUE</code>	Unique user handle.
<code>avatar</code>	<code>TEXT</code>	Yes	-	URL of the profile avatar image.
<code>bio</code>	<code>TEXT</code>	Yes	<code>DEFAULT ''</code>	Short bio or custom description.
<code>role</code>	<code>TEXT</code>	No	<code>CHECK (role IN ('admin', 'member'))</code>	Administrative level within group scope.
<code>family_id</code>	<code>UUID</code>	Yes	<code>REFERENCES family_groups(id)</code>	Currently active family group ID.
<code>updated_at</code>	<code>TIMESTAMPTZ</code>	Yes	<code>DEFAULT now()</code>	Last update timestamp.

2. Table: `family_groups`

Defines family household containers.

Table 3: Family Groups Database Schema

FIELD	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
<code>id</code>	<code>UUID</code>	No	<code>PRIMARY KEY</code>	Unique ID of the family group.
<code>family_code</code>	<code>TEXT</code>	No	<code>UNIQUE</code>	Human-readable ID (e.g. MSF-A7D8E2).
<code>name</code>	<code>TEXT</code>	No	-	Display name of the family unit.
<code>description</code>	<code>TEXT</code>	Yes	<code>DEFAULT ''</code>	Summary description of the household.
<code>avatar_url</code>	<code>TEXT</code>	Yes	-	URL to family group avatar image.
<code>created_by</code>	<code>UUID</code>	Yes	<code>REFERENCES profiles(id)</code>	Creator profile link.
<code>invite_token</code>	<code>TEXT</code>	No	<code>UNIQUE</code>	Secure UUID string for joins.
<code>created_at</code>	<code>TIMESTAMPZ</code>	No	<code>DEFAULT now()</code>	Date of group initialization.

3. Table: `transactions`

Contains financial entries (incomes, expenses, and savings allocations).

Table 4: Transactions Database Schema

FIELD	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
id	UUID	No	PRIMARY KEY	Transaction ID.
amount	NUMERIC	No	CHECK (amount > 0)	Monetary value.
category	TEXT	No	-	Grouping tag (e.g. Food, Bills).
type	TEXT	No	CHECK (type IN ('income', 'expense', 'savings', 'loan'))	Ledger type.
date	DATE	No	-	Transaction date.
notes	TEXT	Yes	-	Free-text notes or transaction memo.
member_id	UUID	No	REFERENCES profiles (id)	Owner profile ID.
proof_url	TEXT	Yes	-	Link to receipt file in Storage.
source	TEXT	No	DEFAULT 'manual'	Source of entry (manual or SMS).
bank_name	TEXT	Yes	-	Detected financial institution.
merchant_name	TEXT	Yes	-	Merchant matching string.
sms_confidence	NUMERIC	Yes	-	Probability score of SMS parse engine.
sms_reference	TEXT	Yes	UNIQUE	Deduplication tracking key.
created_at	TIMESTAMPZ	No	DEFAULT now()	Server insert timestamp.

CHAPTER 13

Core User Flows

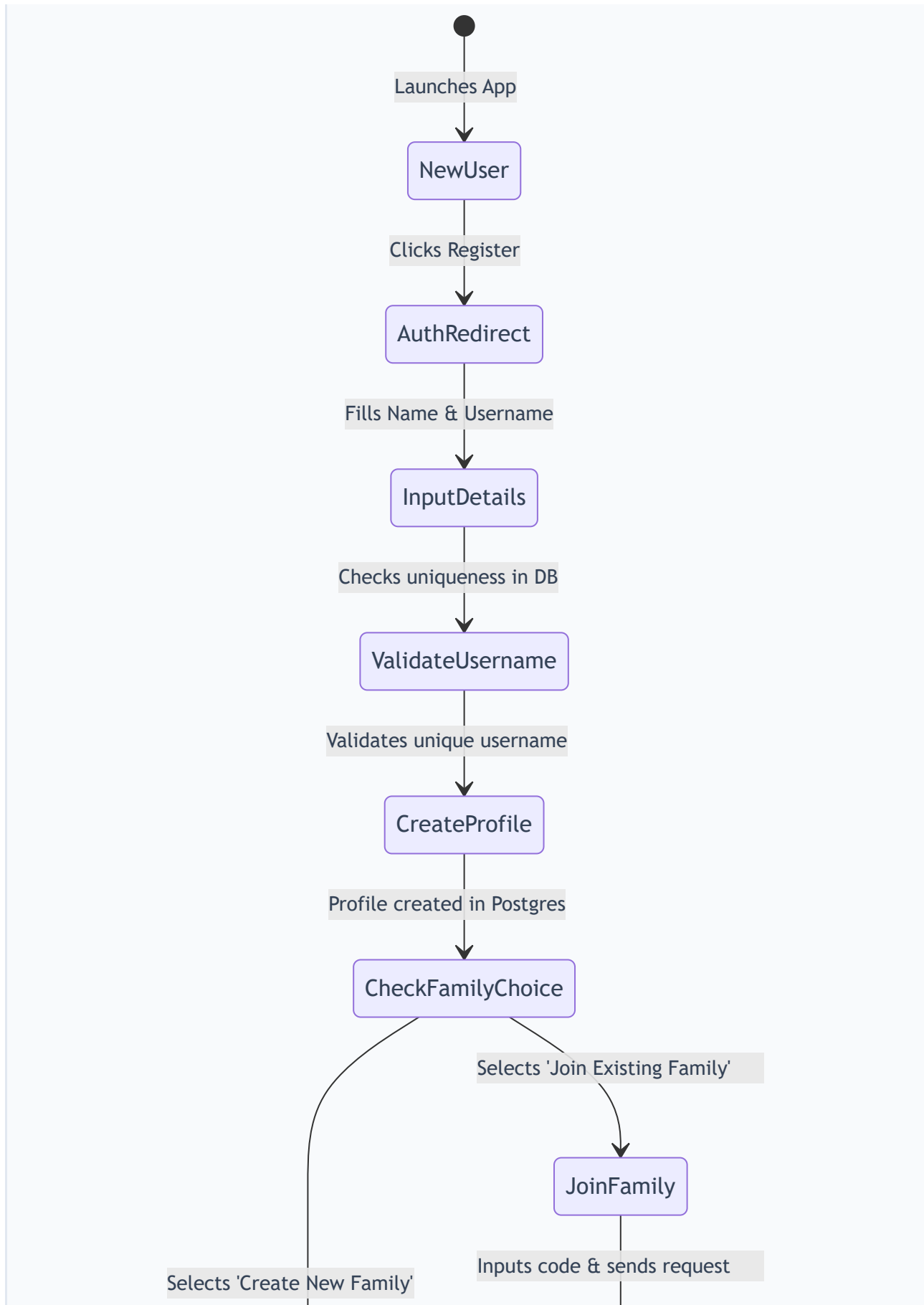
Onboarding, Document OCR, and Location Sharing Flows

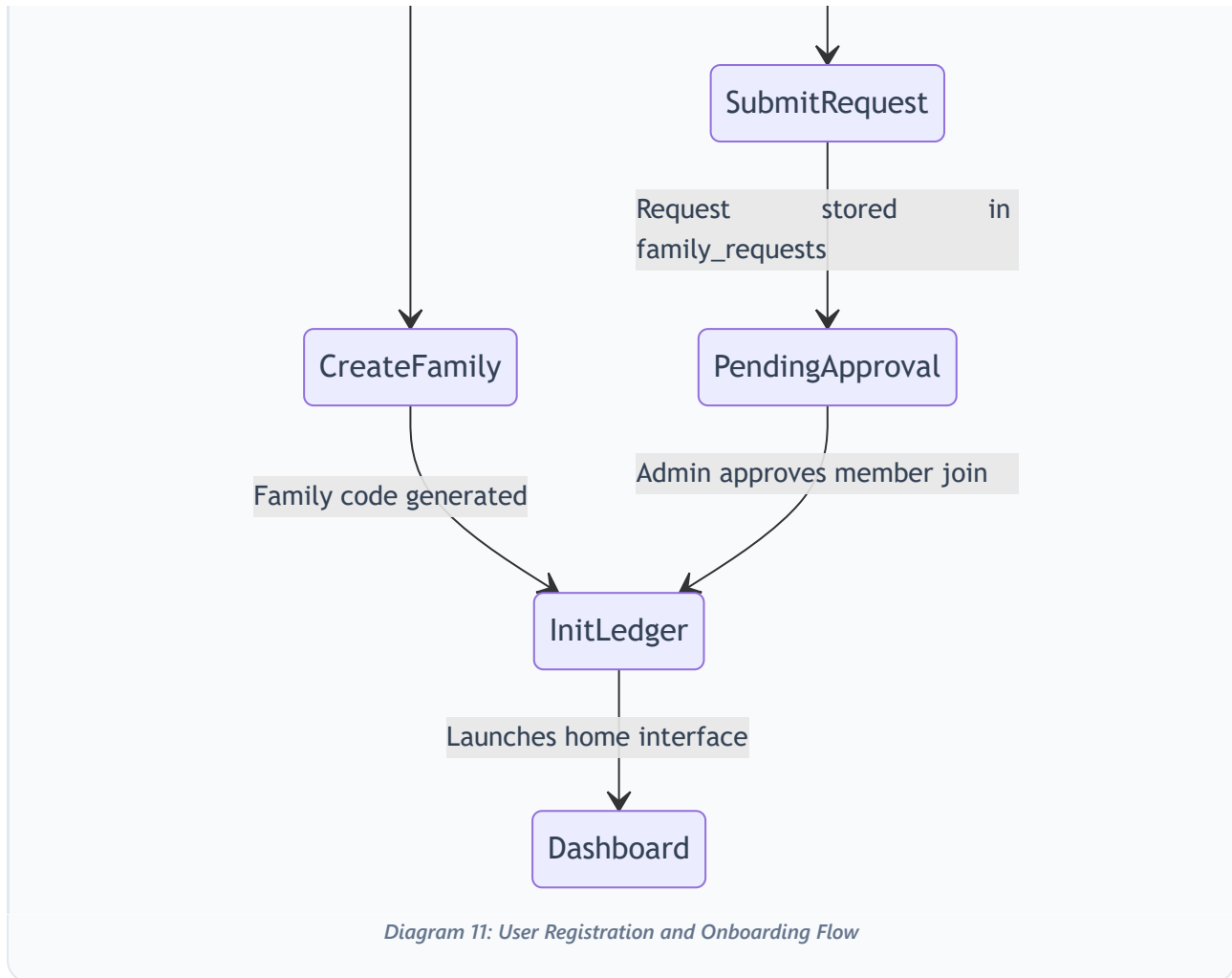
Summary: Displays state transitions and process workflows for onboarding and document vault ingestion.

1. Onboarding & Profile Creation Flow

Shows the steps required for a new user to register and join a family group.



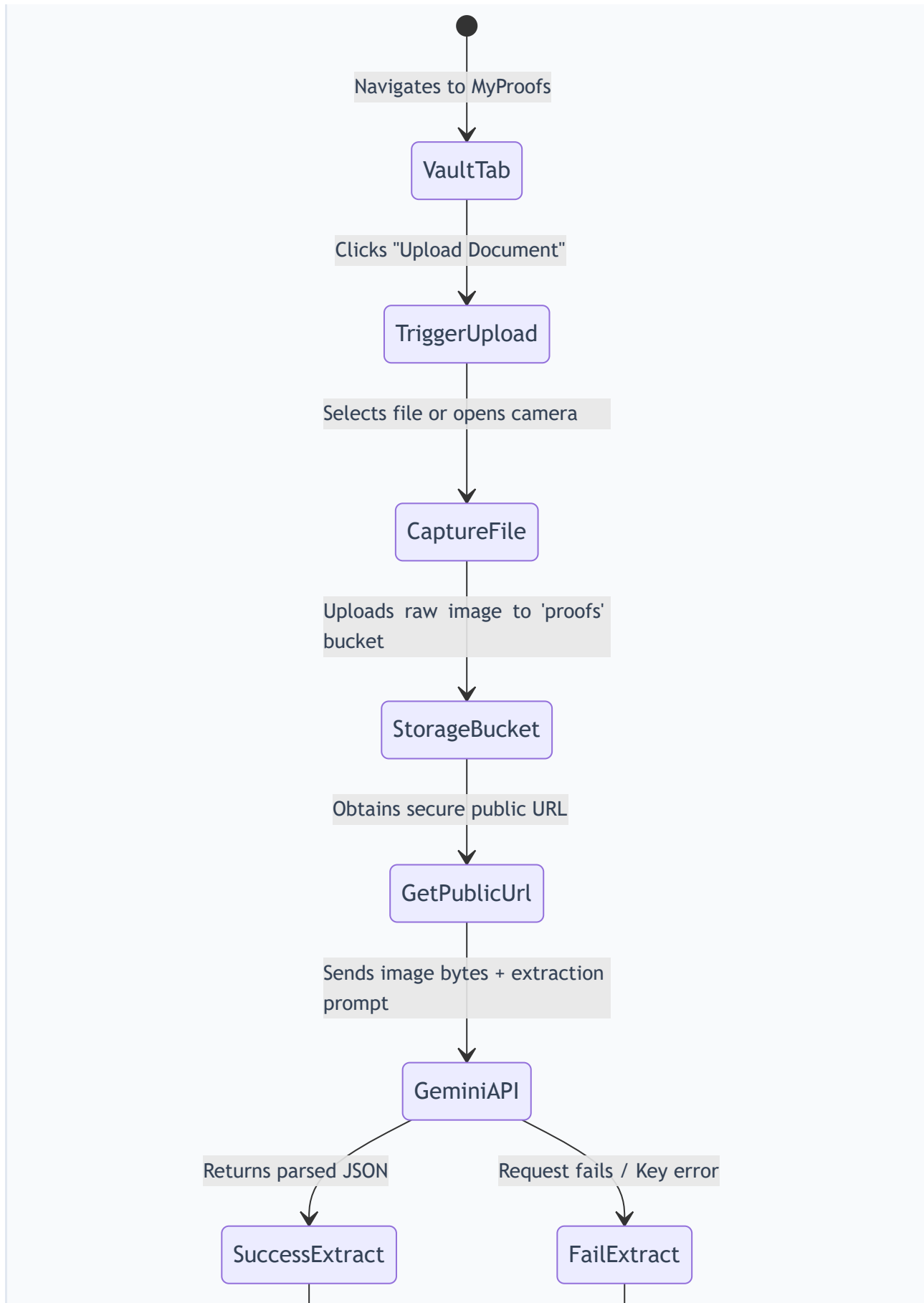


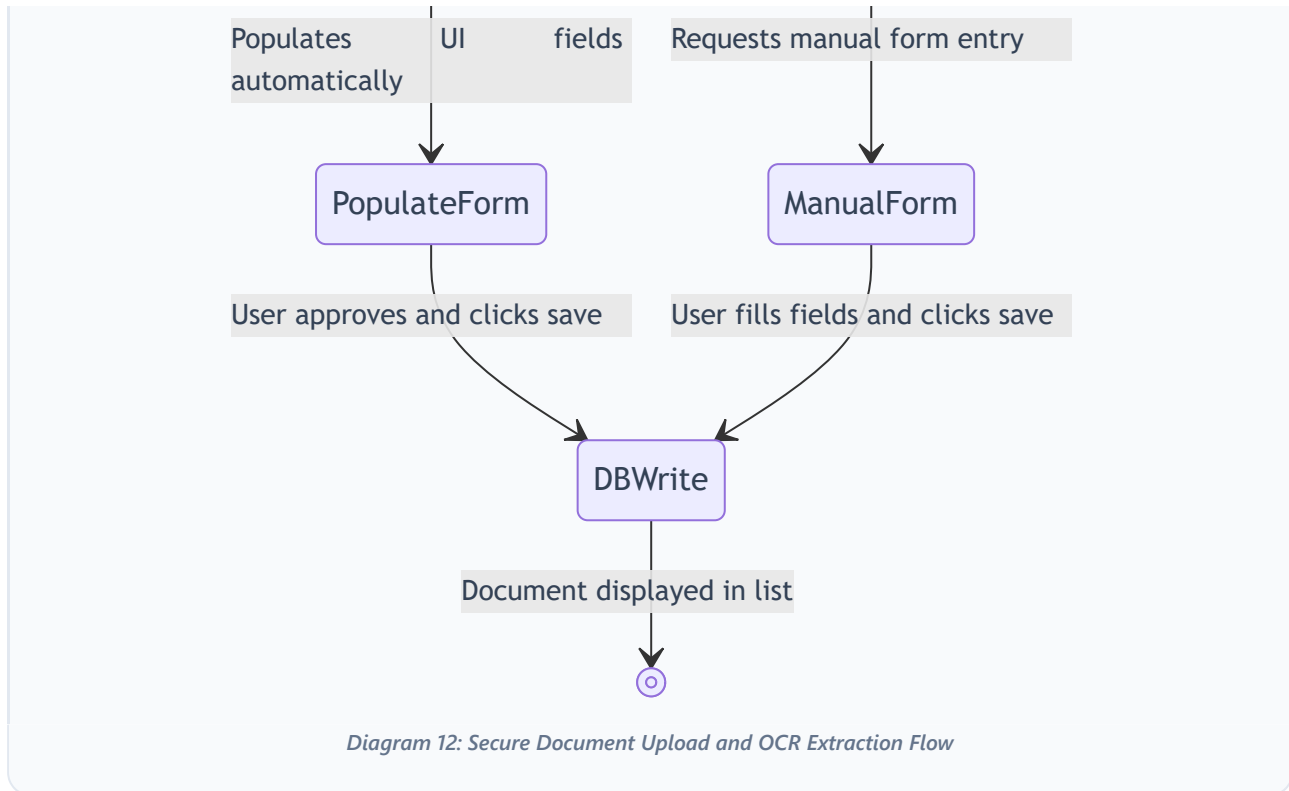


2. Document OCR Ingestion Flow

Shows how a receipt or invoice is uploaded and processed using Gemini.







CHAPTER 14

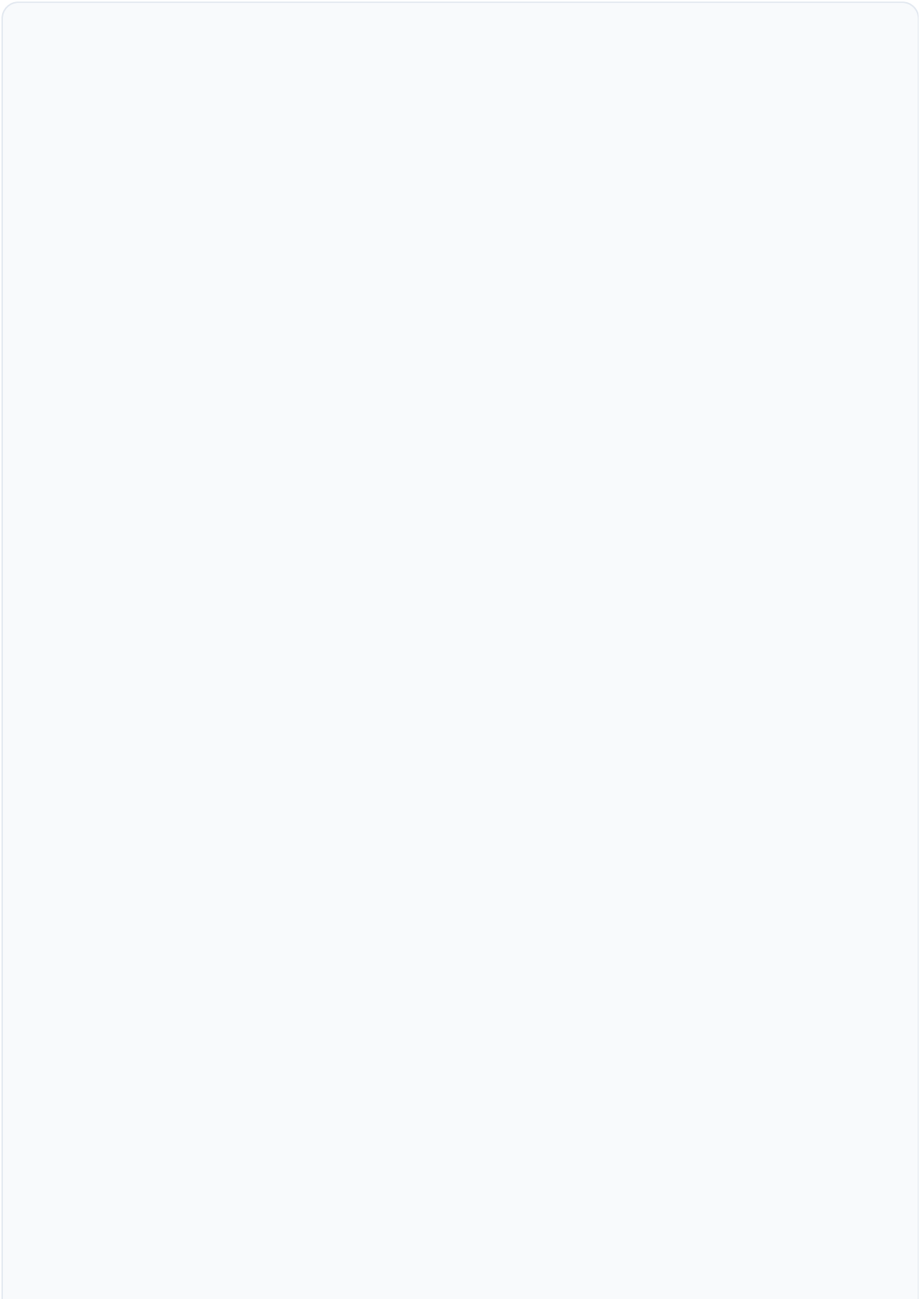
Application Lifecycle Flow

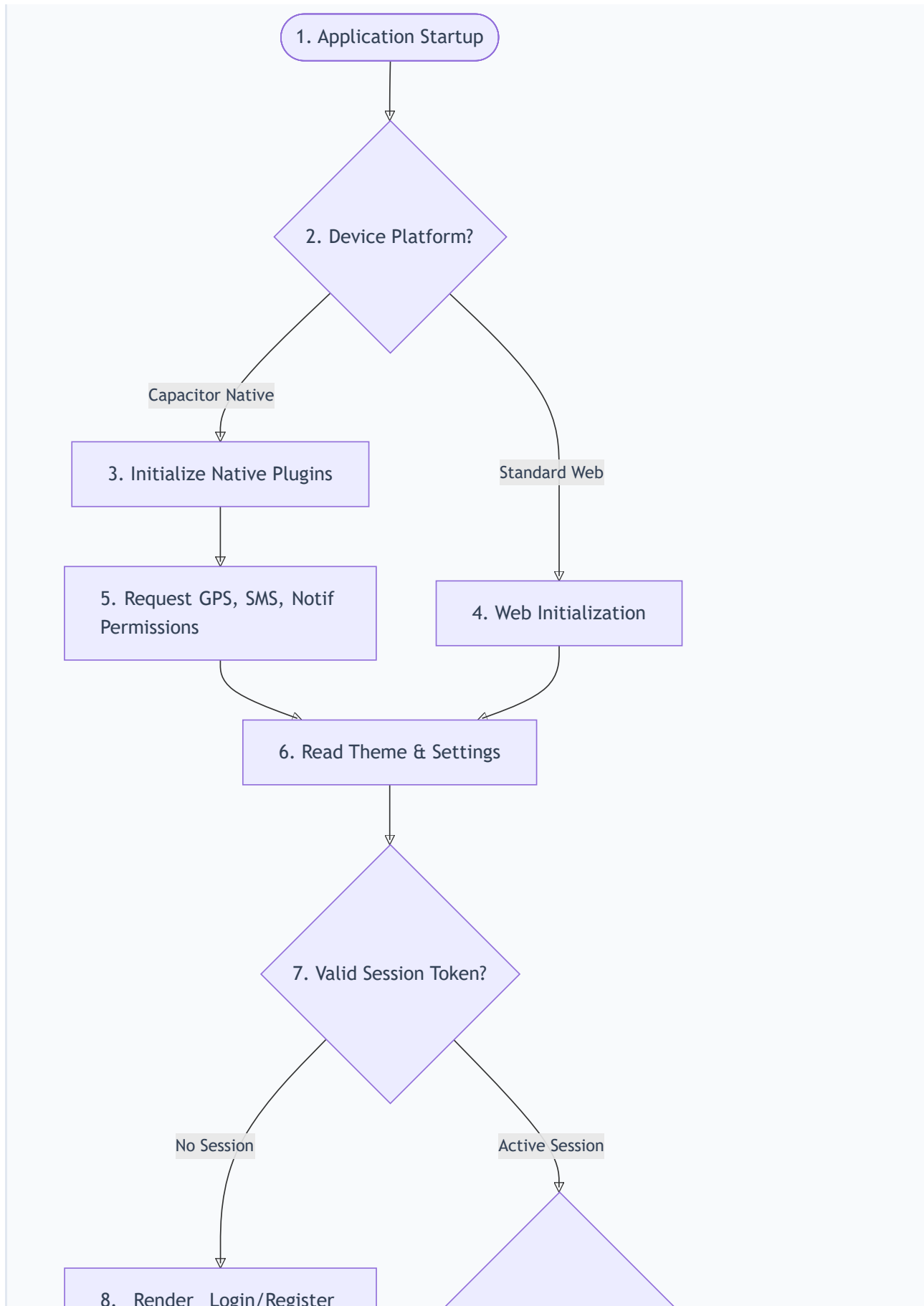
Application Lifecycle, State Transitions, and Background Execution

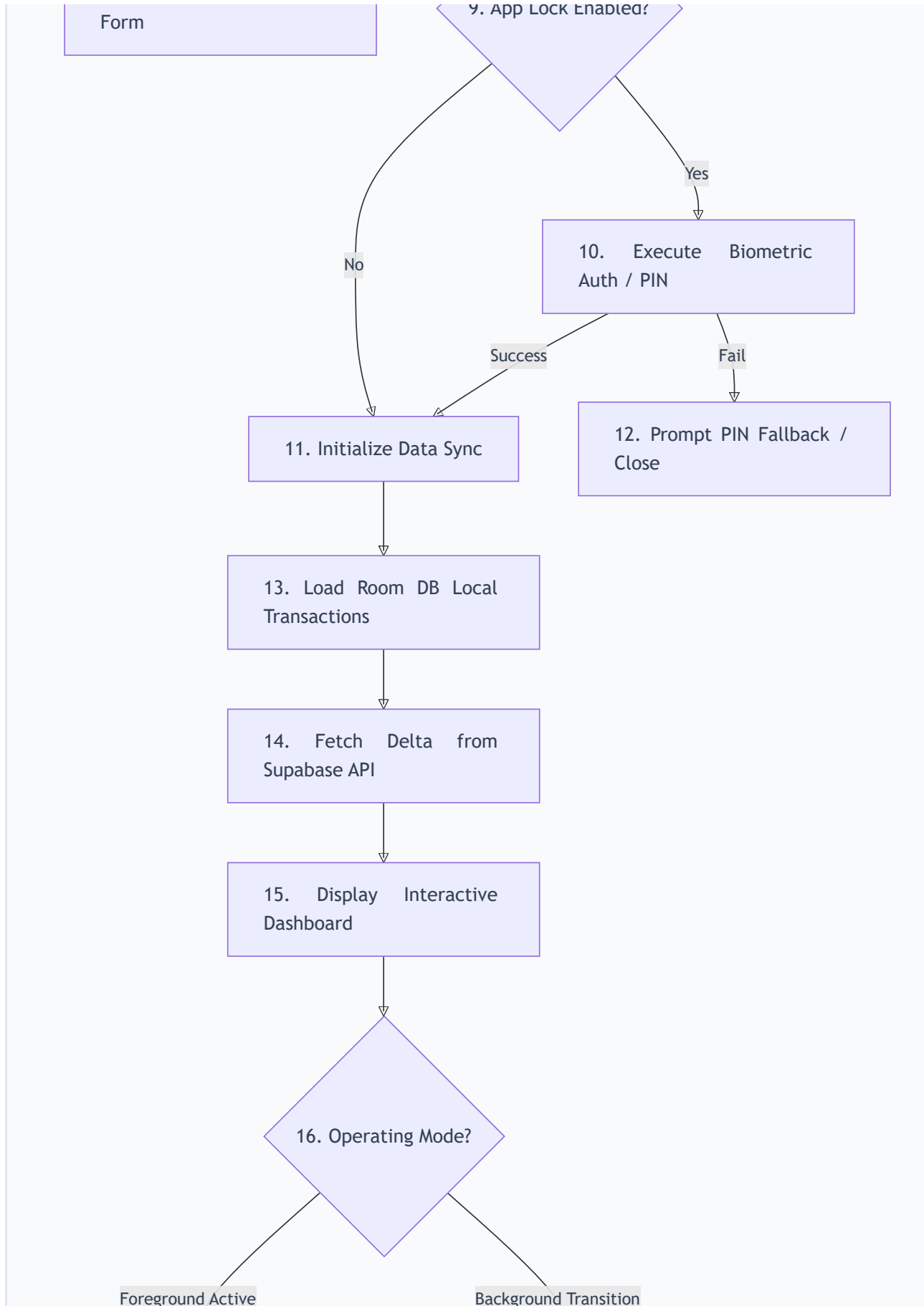
Summary: Details the application lifecycle, from startup checks through to background execution.

1. Application Runtime States

Shows the initialization steps, biometric checks, and active synchronization processes.







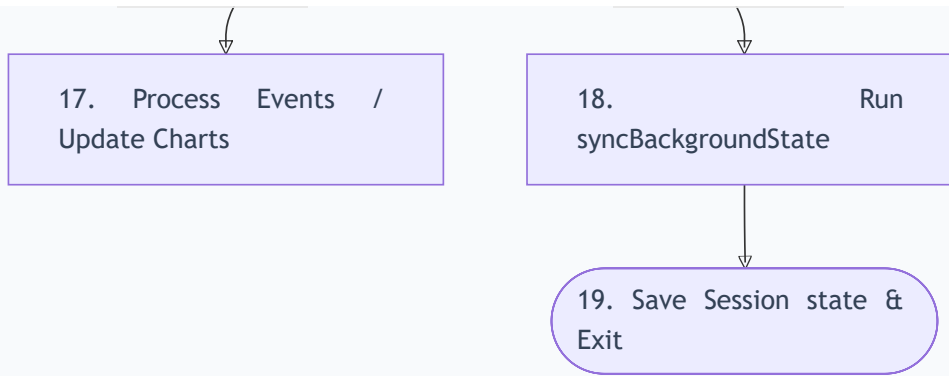


Diagram 13: Application Runtime State Transitions

i Platform Lifecycle Event

In mobile applications, the `appStateChange` event triggers data synchronization automatically when the app is minimized.

CHAPTER 15

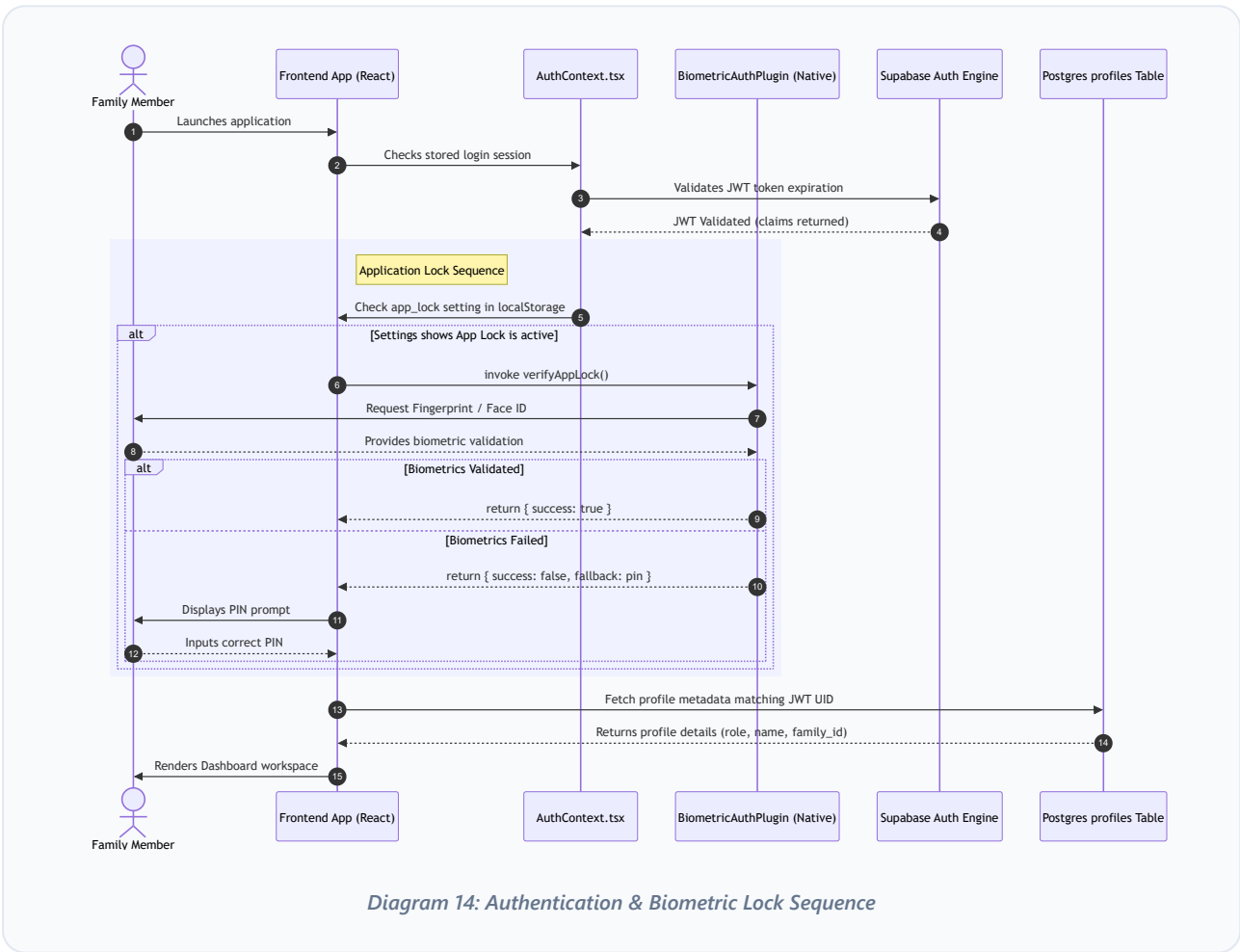
Authentication & Lock Sequence

JWT Validations, Session Checks, and Biometric Lock Integrations

Summary: UML sequence showing the steps for user authentication and biometric access validation.

1. Authentication Security Sequence

Describes how the client app interacts with native biometrics and remote databases.



CHAPTER 16

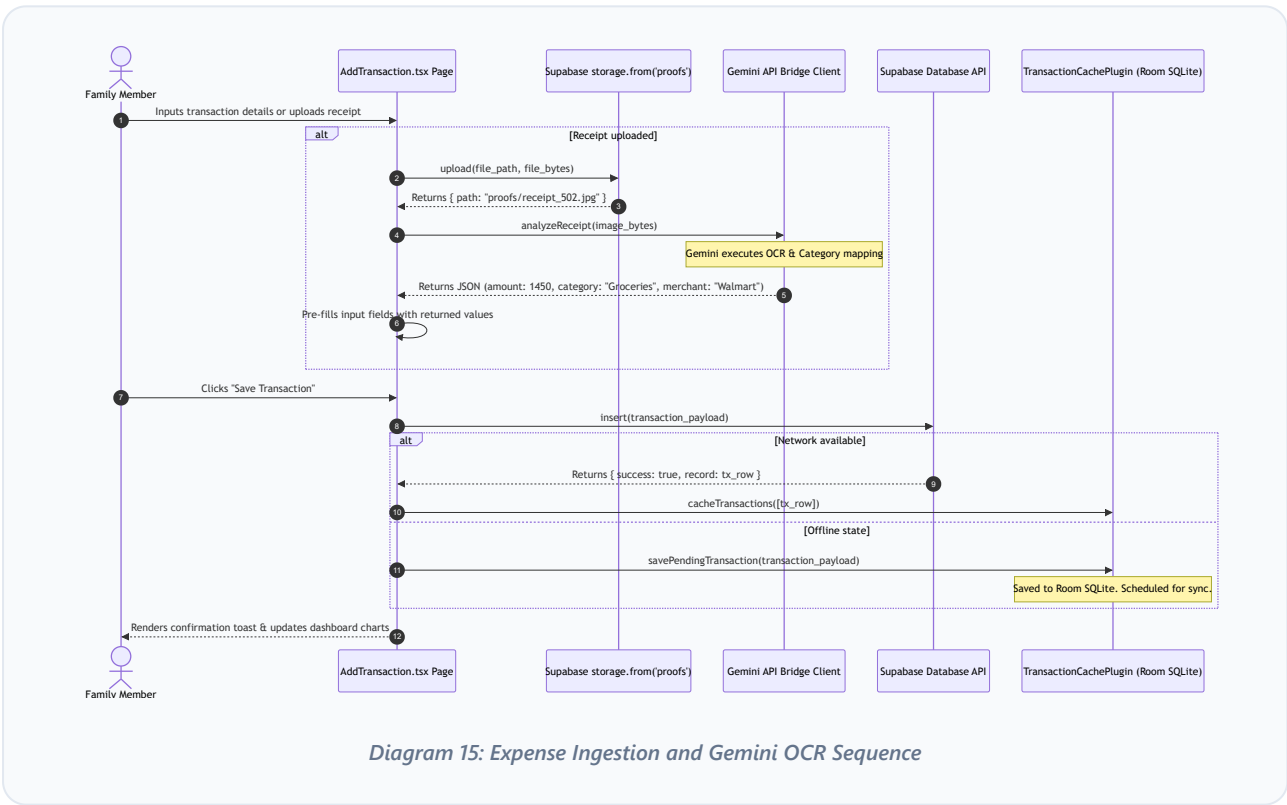
Expense & OCR Ingestion Sequence

Manual entry, Storage uploads, Gemini parsing, and Local Caching

Summary: Details the receipt processing pipeline, showing interactions between the client, Storage, and the Gemini API.

1. Ingestion Sequence

Steps for uploading and parsing receipt images.



CHAPTER 17

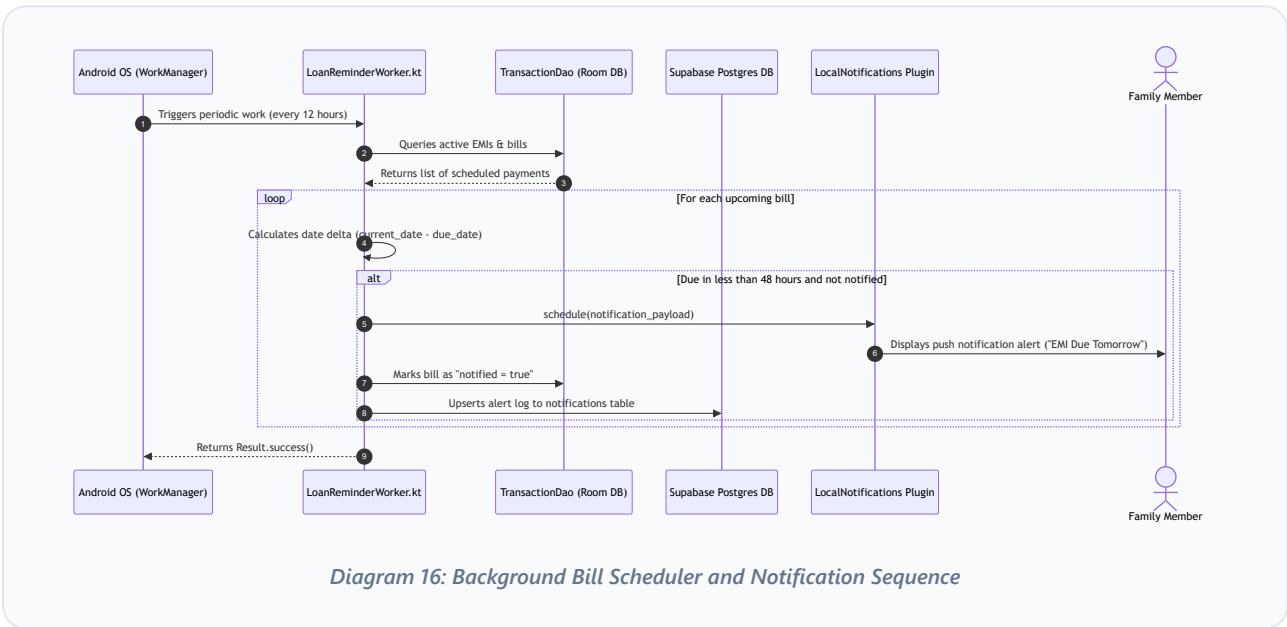
Bill Scheduler & Alert Sequence

WorkManager triggers, Database checks, and Local Notification Dispatch

Summary: Explains how the native scheduler triggers notifications for upcoming bills and EMIs.

1. Native Scheduling Sequence

Details the background execution steps that monitor payment schedules.



CHAPTER 18

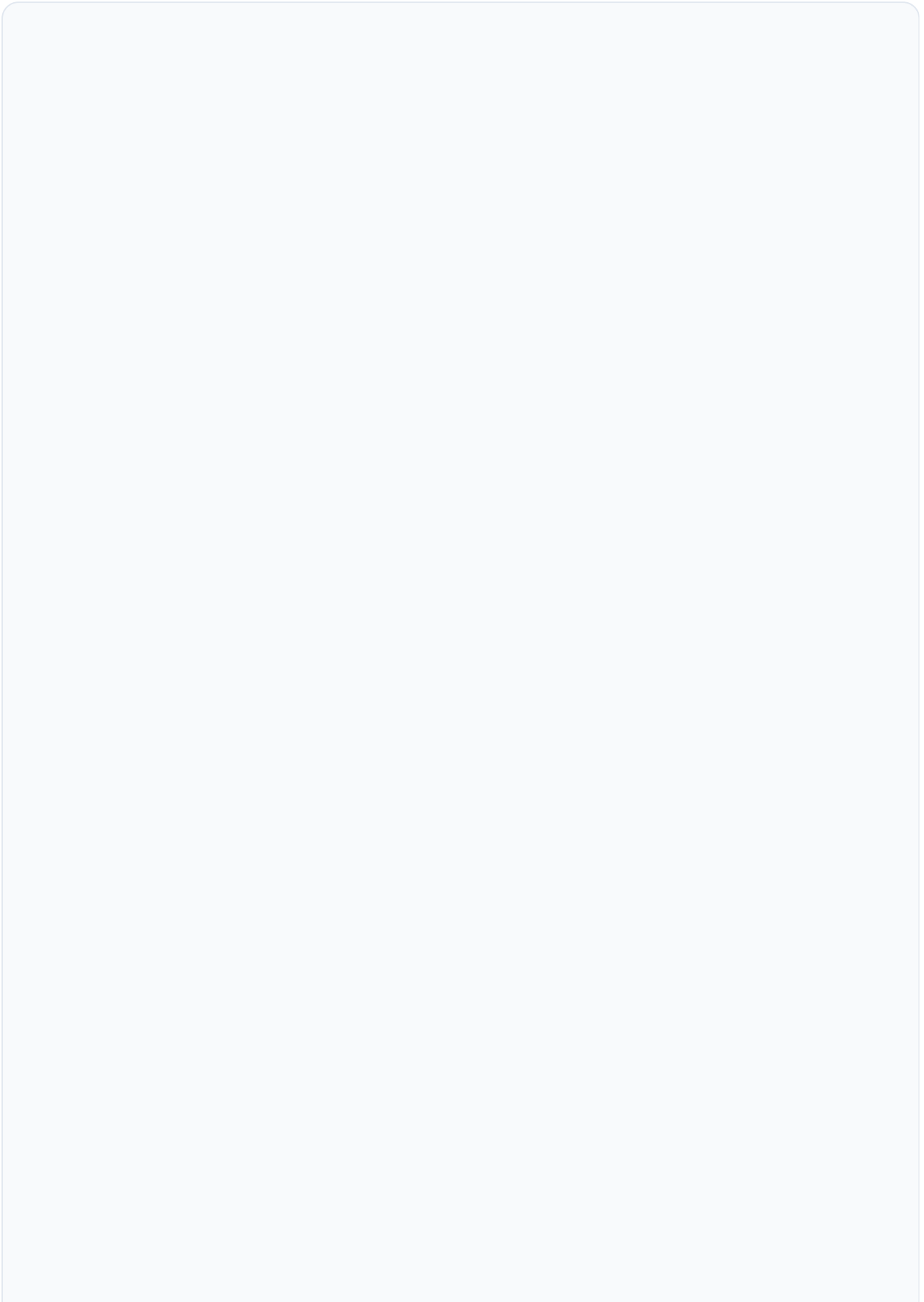
AI Assistant Reasoning Sequence

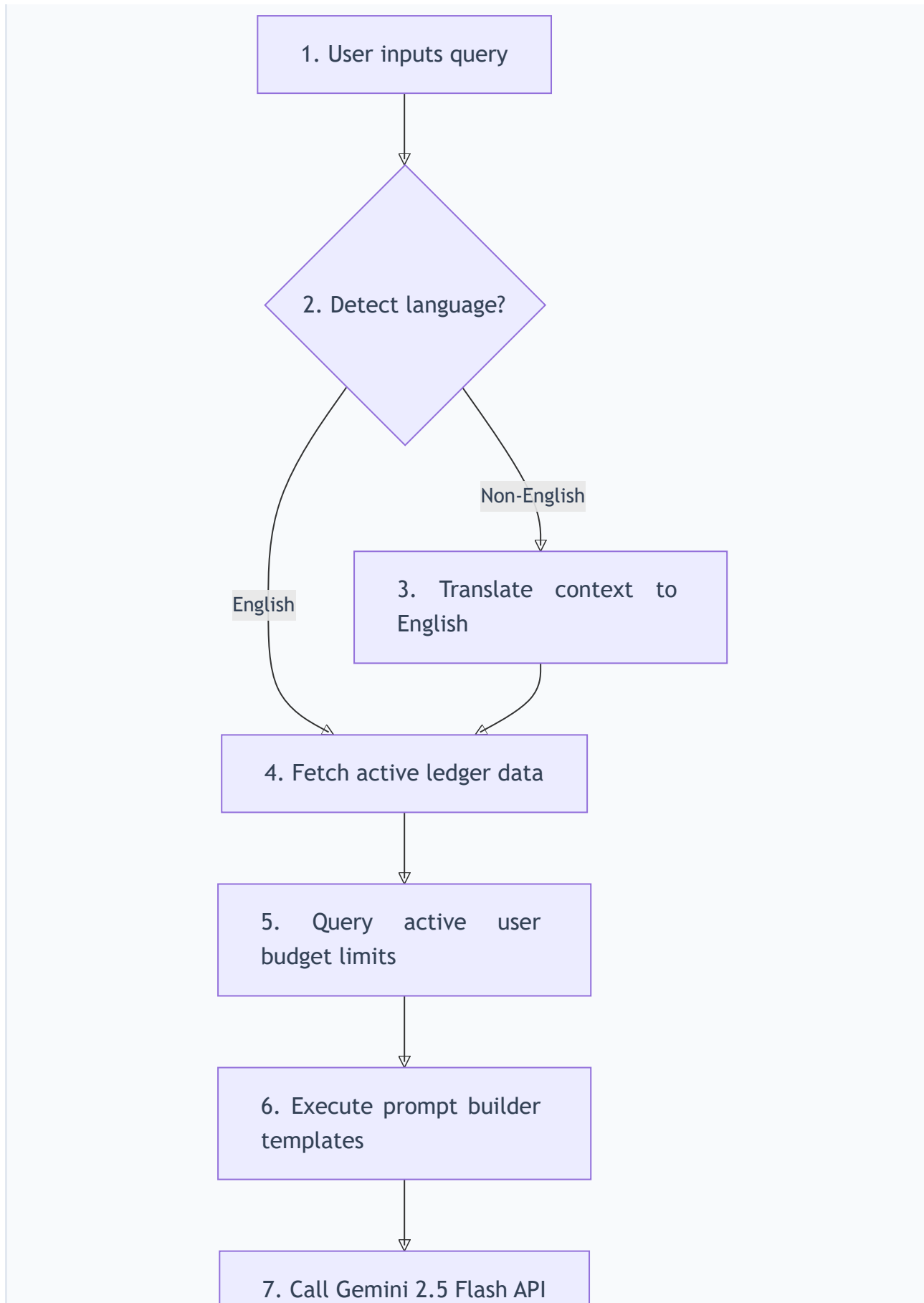
Language processing, Context builds, Gemini APIs, and Output translation

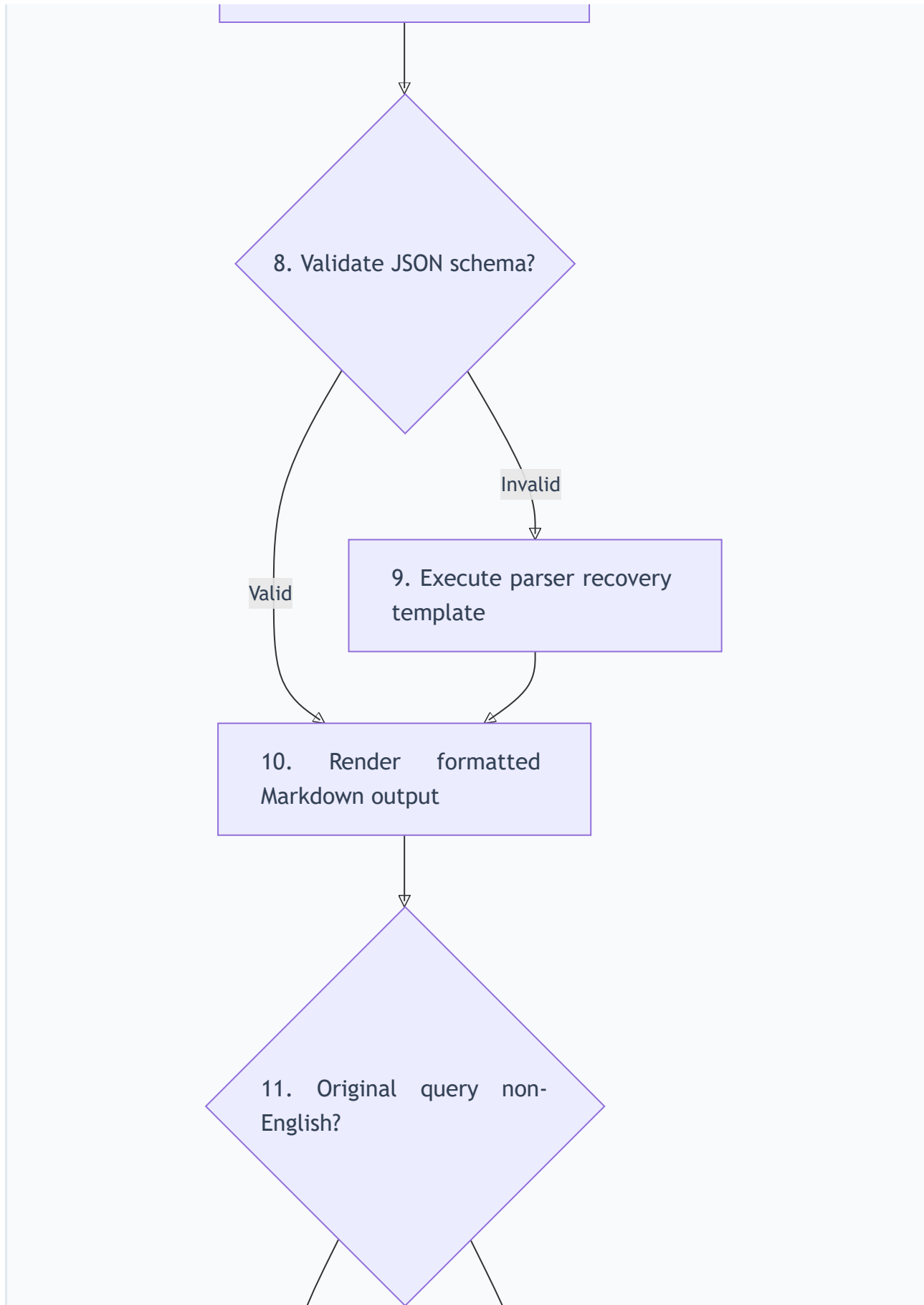
Summary: Details the query parsing and context generation steps used by the AI assistant.

1. Inquiry Pipeline

Shows the processing steps for incoming user questions.







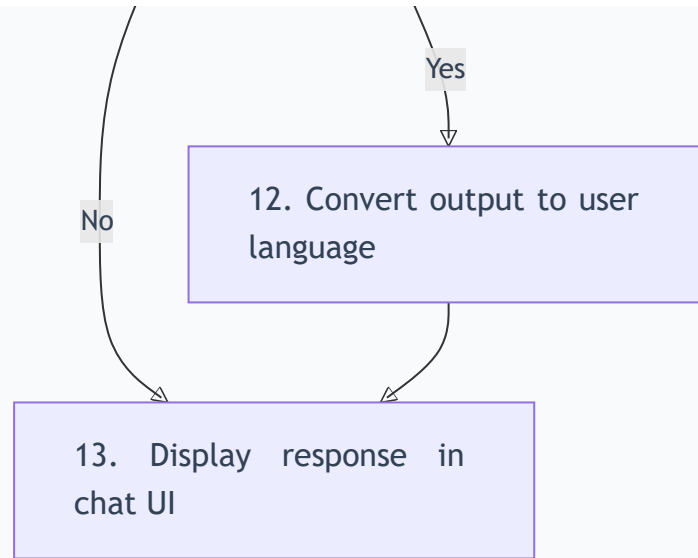


Diagram 17: AI Assistant Query Processing Pipeline

CHAPTER 19

Secure Document Ingestion Sequence

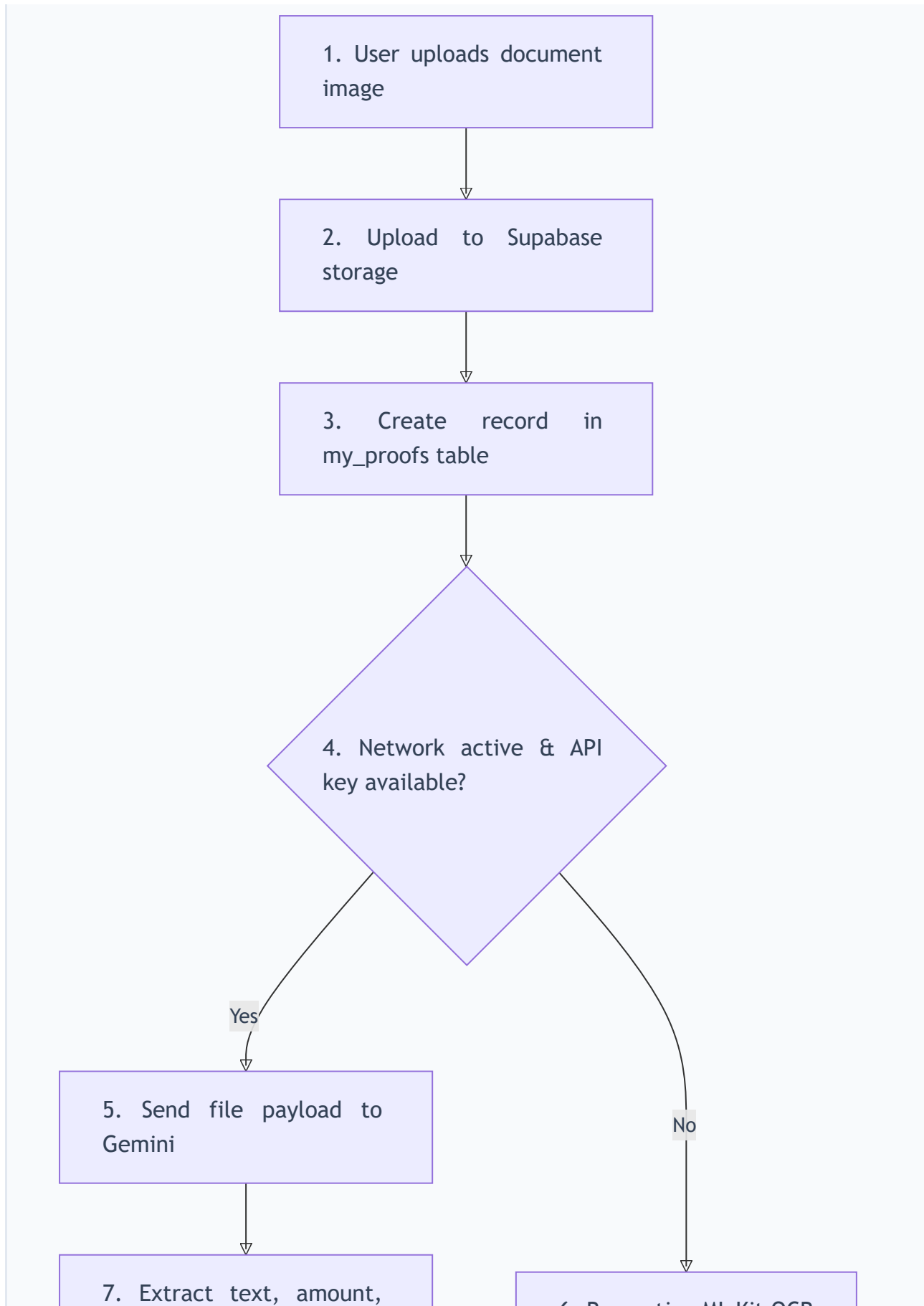
File Compression, Supabase Storage, and Metadata Extraction

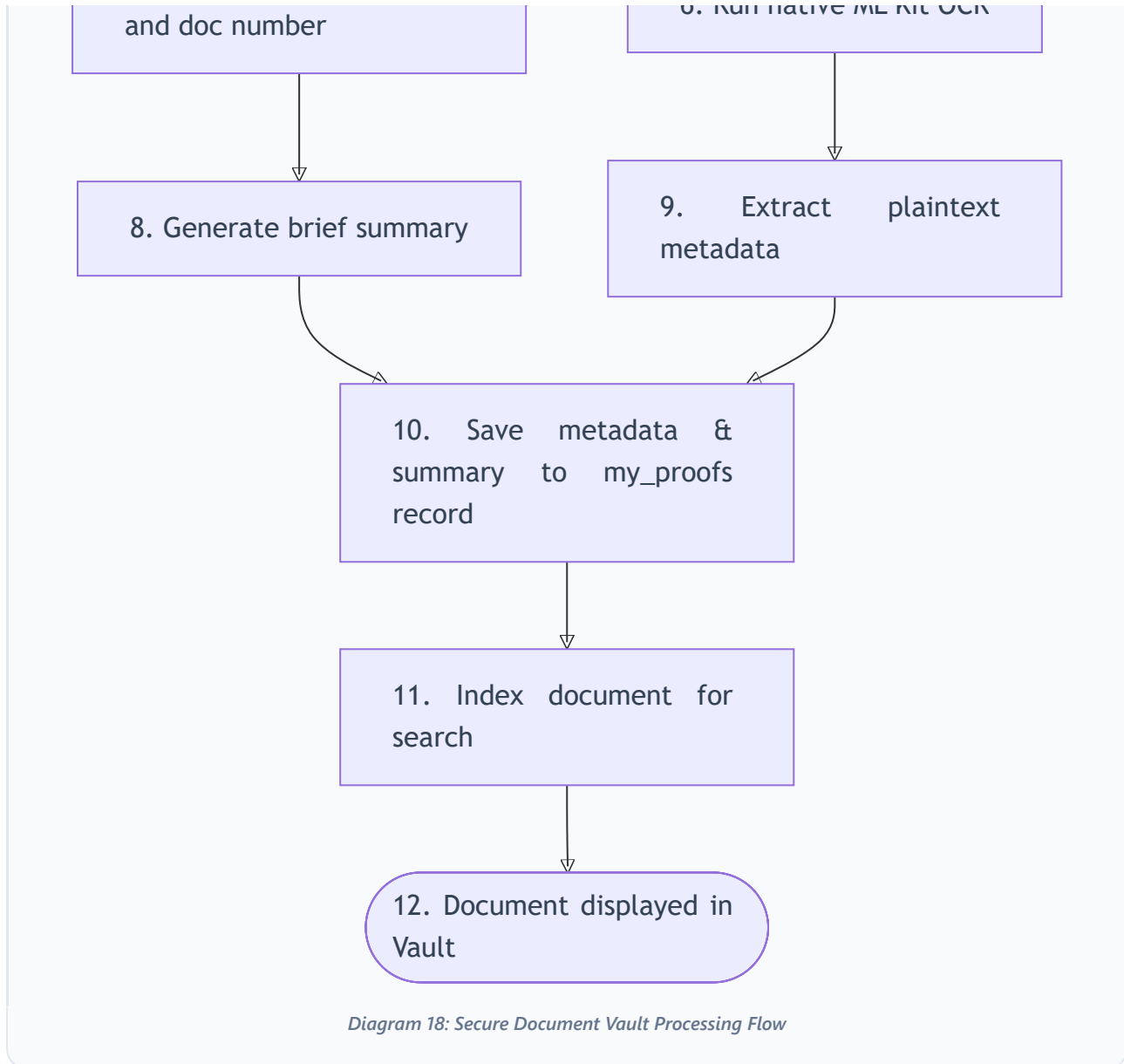
Summary: Outlines the document ingestion process, detailing the storage path and search indexing.

1. Document Vault Flow

Shows the steps to upload and index document receipts.







CHAPTER 20

Vehicle Service & Alert Sequence

Database columns, service schedules, and alert triggers

Summary: Details the database design and notifications schedule used to track vehicle maintenance.

1. Service Alert Pipeline

Describes how service date thresholds trigger notifications.

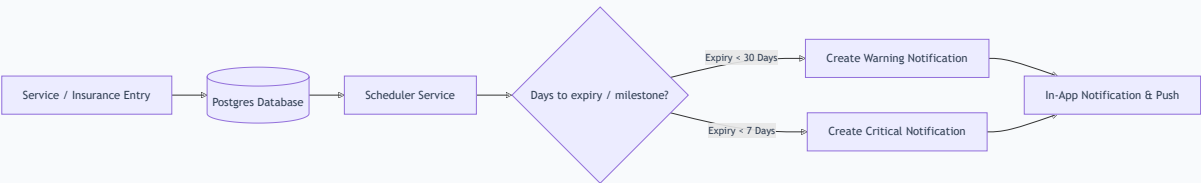


Diagram 19: Vehicle Service Alert Logic

2. Table: `vehicles`

Stores vehicle details.

Table 5: Vehicles Database Schema

FIELD	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PRIMARY KEY	Unique vehicle ID.
name	TEXT	-	Display name (e.g. Model Y).
license_plate	TEXT	UNIQUE	Registration number.
purchase_date	DATE	-	Purchase date.
family_id	UUID	REFERENCES family_groups (id)	Link to family container.

CHAPTER 21

Medication Reminder Sequence

AlarmManager registrations, heads-up prompts, and adherence logging

Summary: Details the scheduling engine and database logs used to manage medication compliance.

1. Medication Reminder Pipeline

Shows the native scheduling sequence and user compliance logging.

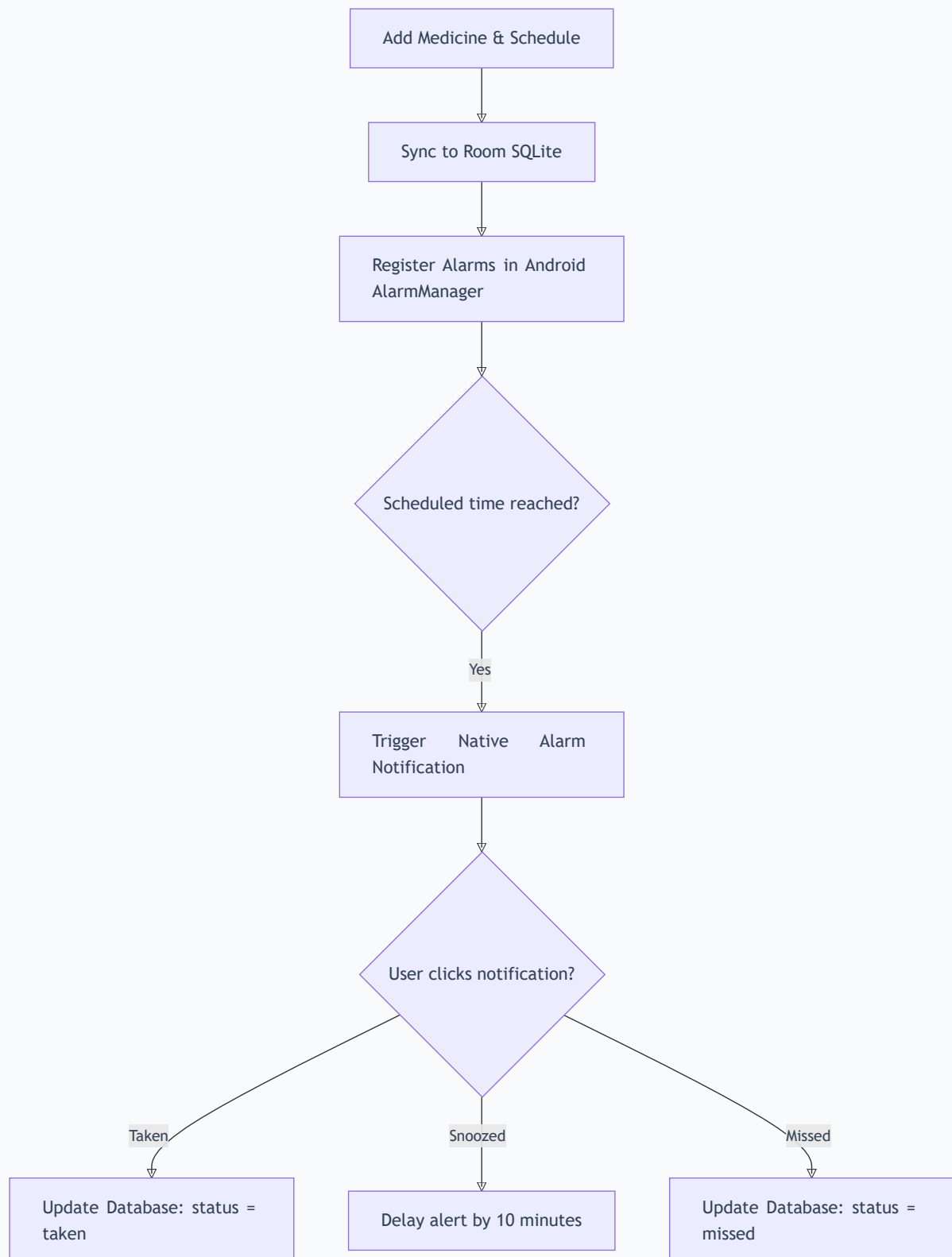


Diagram 20: Medication Reminder Schedule and Compliance Flow

CHAPTER 22

REST API Specifications

Request Headers, Request Bodies, Responses, and Error Schemes

Summary: Complete endpoint definitions for authentication, family groups, and transaction ledgers.

1. Authentication Endpoints

POST `/auth/v1/signup`

Registers a new user profile.

- **Request Headers:** `http Content-Type: application/json apikey: SUPABASE_ANON_KEY`
- **Request Body:** `json { "email": "user@example.com", "password": "SecurePassword123", "data": { "name": "ArulPrakash", "username": "arul_p" } }`
- **Response (Status 200 OK):** `json { "access_token": "eyJhbGciOi...", "token_type": "bearer", "expires_in": 3600, "user": { "id": "e3b0c442-98fc-11eb-a8b3-0242ac130003", "email": "user@example.com" } }`

2. Transactions Endpoints

GET `/rest/v1/transactions`

Retrieves transactions for the active family group.

- **Request Headers:** `http Authorization: Bearer USER_JWT_TOKEN Range: 0-9`
- **Query Parameters:**
- `member_id`: Filter by owner profile ID (e.g. `eq.e3b0c442-98fc-11eb-a8b3-0242ac130003`)
- **Response (Status 200 OK):** `json [ { "id": "1d8b9f42-4b2a-4a8e-bc5d-6c12e9cf41d2", "amount": 2500.00, "category": "Utilities", "type": "expense", "date": "2026-07-05", "notes": "Electric Bill", "member_id": "e3b0c442-98fc-11eb-a8b3-0242ac130003" } ]`

CHAPTER 23

Security & Threat Architecture

STRIDE Threat Modeling, RLS Rules, and OWASP Mitigations

Summary: Outlines the security architecture, data-at-rest encryption, and threat mitigations.

1. STRIDE Threat Model

Table 6: STRIDE Threat Model Mitigations

THREAT CATEGORY	POTENTIAL SYSTEM RISK	MITIGATION STRATEGY
Spoofing	Unauthorized user accesses another family's ledger.	Mandatory JWT validation on every query. Supabase RLS verification.
Tampering	Intercepting data in transit to modify balances.	Enforcement of TLS 1.3. Rejecting non-HTTPS client API requests.
Repudiation	User denies performing a transaction or deleting a file.	Audit trail tables logging all deletions and administrative updates.
Information Disclosure	Location leakage or exposing document contents.	Storage access verification based on owner ID folders.
Denial of Service	Flooding backend APIs or Edge functions.	Cloudflare rate-limiting rules. Max query limit checks in Postgres.
Elevation of Privilege	Normal family member tries to view Admin Activity logs.	Role checks (<code>role = 'admin'</code>) enforced in database policies.

2. Row-Level Security (RLS) Policies

Row-level access controls are implemented directly at the database layer:

supabase/migrations/001_user_locations.sql

SQL

```
-- Allow reads only for family members within the same group
CREATE POLICY "Allow family members to view shared transactions"
ON public.transactions FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM public.family_members
    WHERE family_members.family_id = (
      SELECT family_id FROM public.profiles
      WHERE profiles.id = auth.uid()
    )
    AND family_members.user_id = transactions.member_id
  )
);
```

CHAPTER 24

Frontend Core Architecture

Directory Structures, Component Model, Routing, and Lazy Chunks

Summary: Details the react component design, state providers, and code directory structure.

1. Component Module Layout

The source directory structure decouples logic into contexts, custom hooks, and pages.

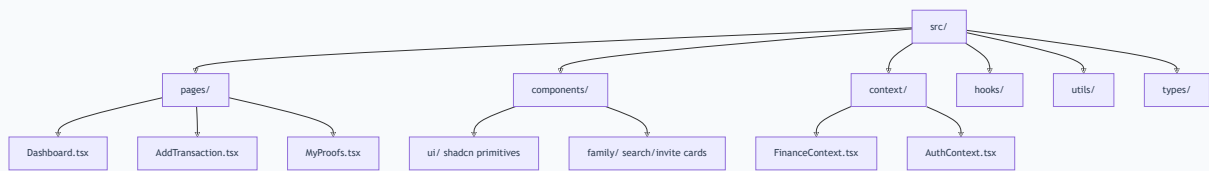


Diagram 21: Frontend Source Directory Organization

2. Core Frontend Patterns

- **Code Splitting:** Vite compiles views (such as `Analytics.tsx` and `Loans.tsx`) into separate chunks to optimize initial page loading.
- **Radix UI Primitives:** Dropdowns and modals are built on Radix primitives to support accessibility requirements.

CHAPTER 25

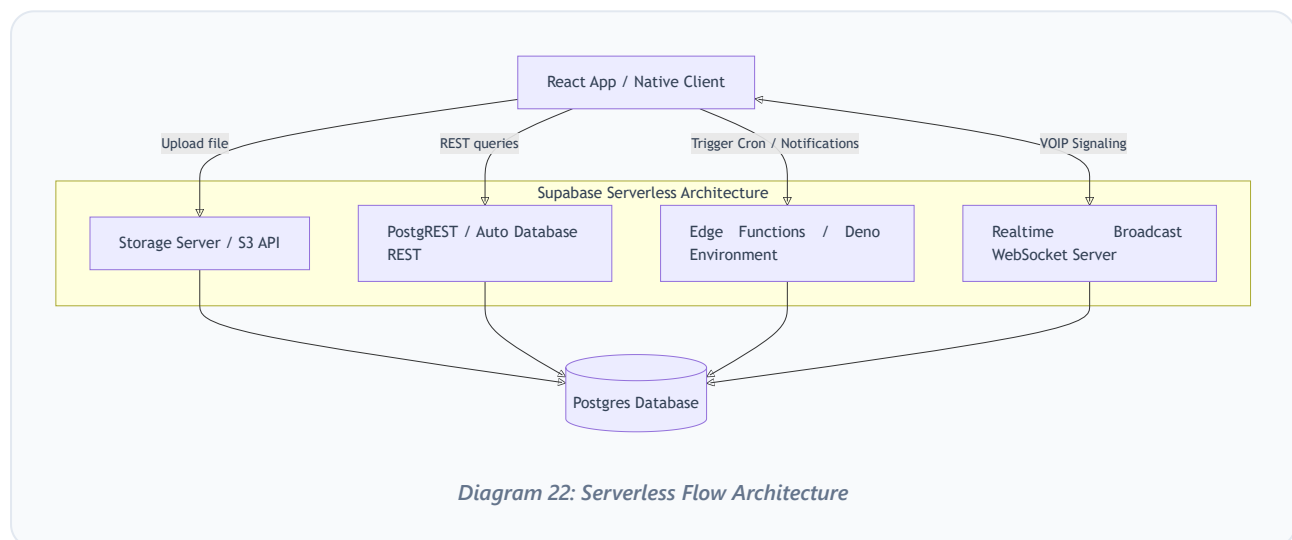
Backend Core Architecture

PostgREST gateways, S3 Storages, Realtime brokers, and Edge functions

Summary: Details the backend components and serverless Edge Function architectures.

1. Serverless Execution Pipelines

Deno runtime components execute serverless tasks.



2. Edge Function Triggers

- **send-notification**: Fired by database webhooks on transaction updates, dispatching push messages via Firebase.
- **daily-rates-push**: PG_Cron executes this function daily to update current currency exchange and physical asset rates.

CHAPTER 26

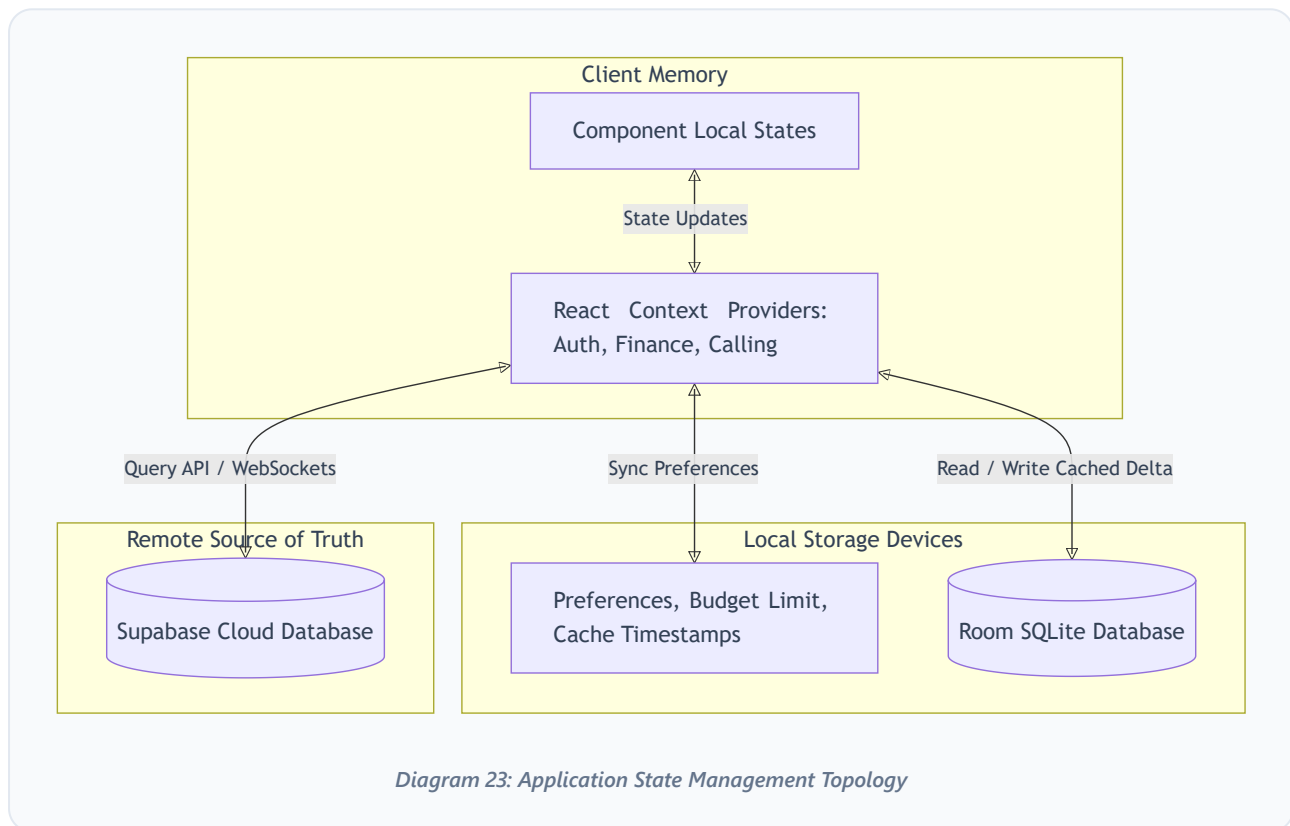
State Management & Persistence

React Contexts, Local Storage, Room Cache, and Synchronization Queue

Summary: Details the synchronization queue, offline storage logic, and local states.

1. Client-to-Cloud State Sync

Tracks data state sync between local storage and remote PostgreSQL.



2. SQLite Cache Deduplication

Incoming transaction records are assigned a unique SMS reference ID. A SQLite constraints checker drops duplicate entries if a collision occurs.

CHAPTER 27

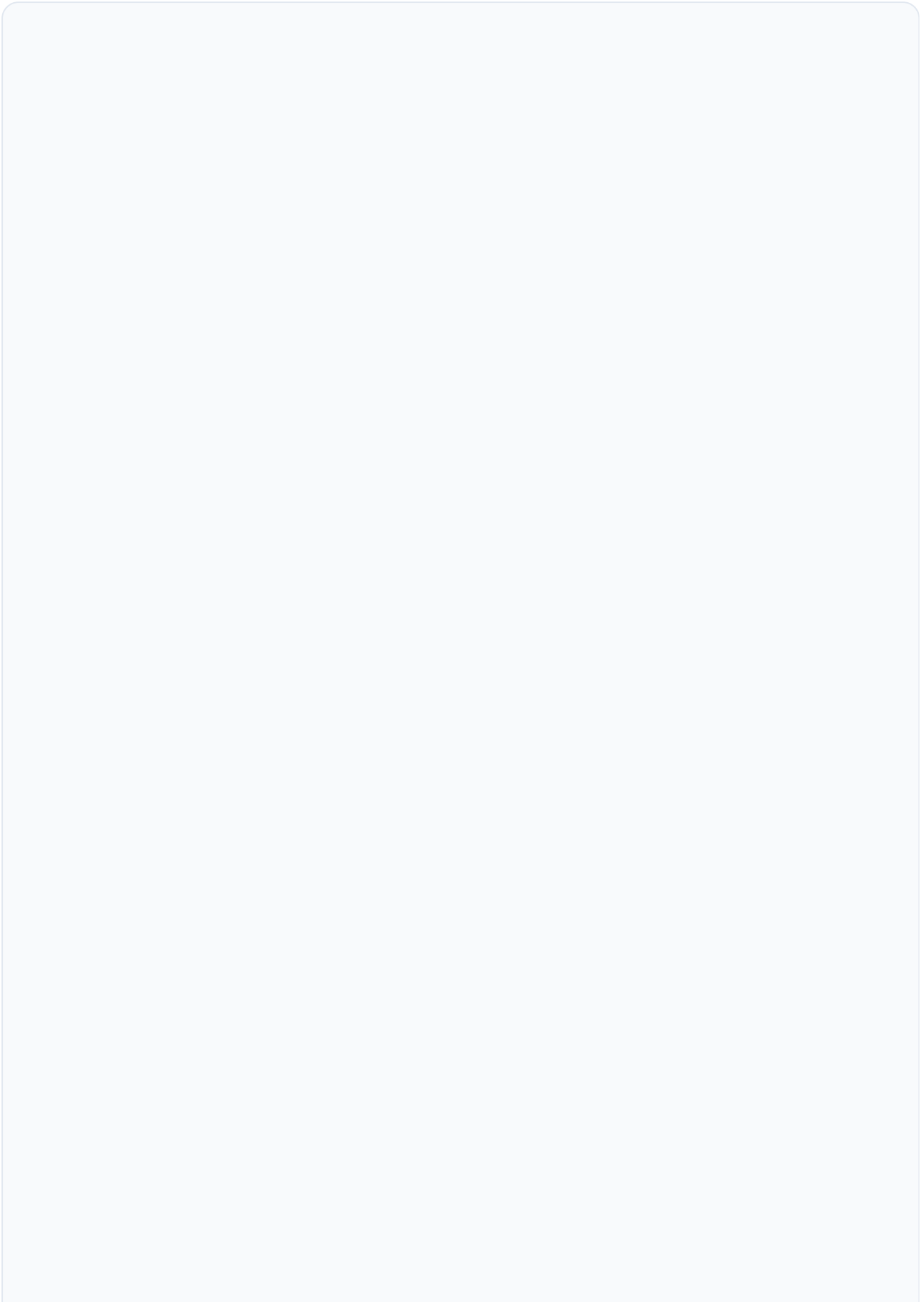
DevOps & CI/CD Pipeline

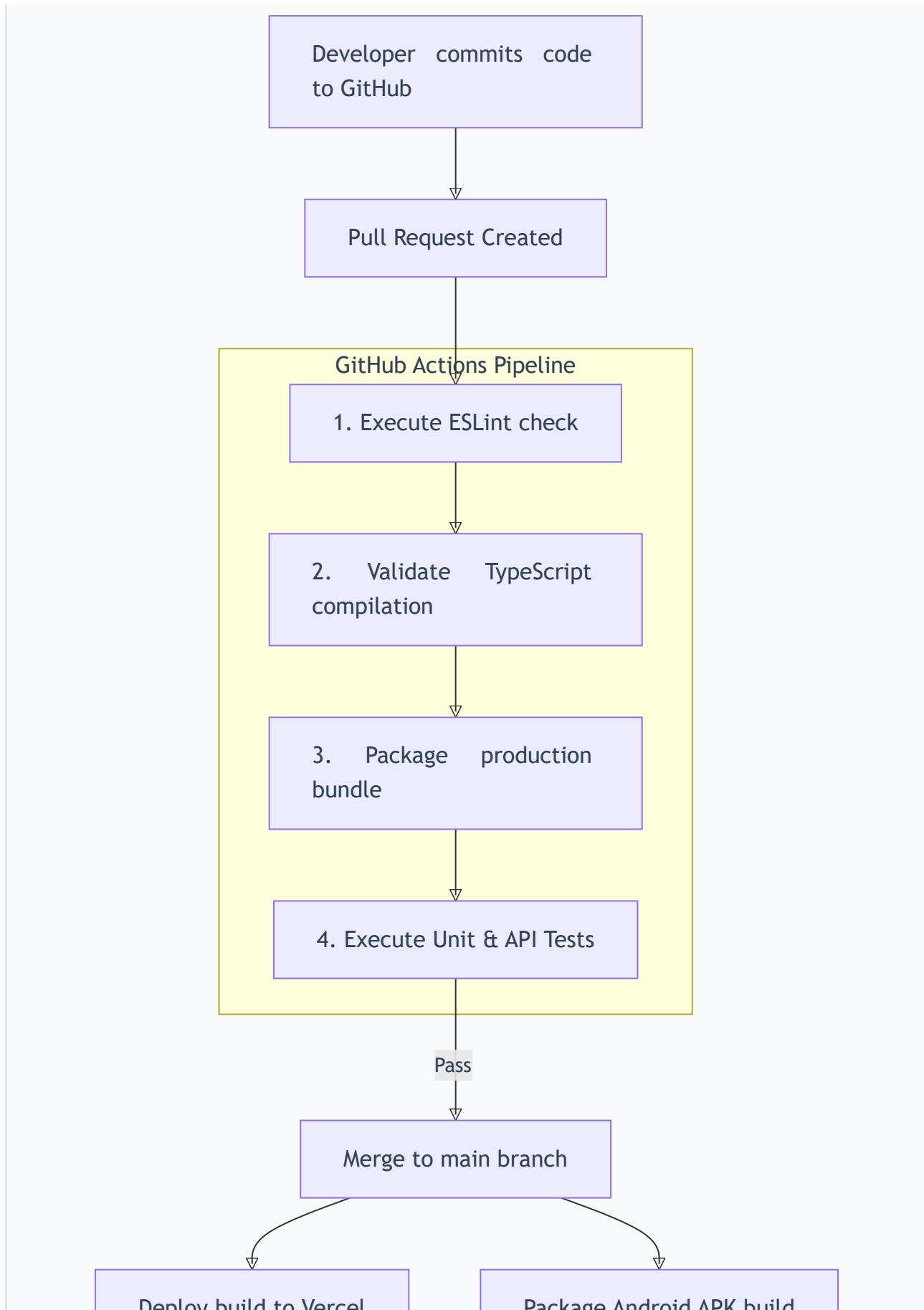
Git workflows, Linting, Testing, and Production deployment

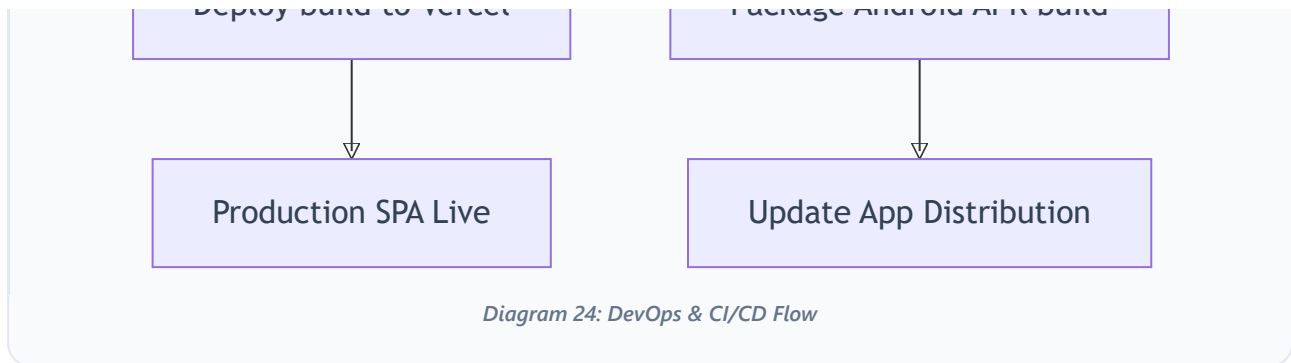
Summary: Outlines the automated integration workflows and build checks used to deploy updates.

1. Automated Integration Workflows

Details the actions triggered by code commits.







2. GitHub Actions Configuration

Validates code compiles successfully before merging:

```
.github/workflows/validate.yml  YAML

name: Build and Validate
on:
  push:
    branches: [main]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Run Build
        run: npm ci && npm run build
```

CHAPTER 28

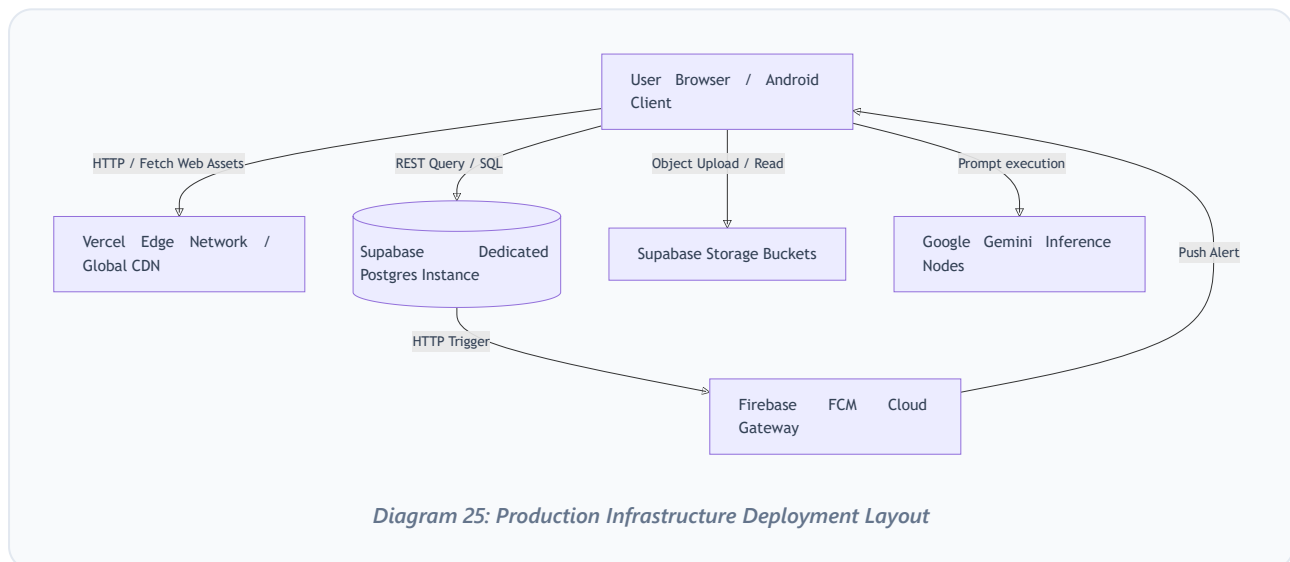
Deployment Architecture Blueprint

Edge Servers, Postgres Nodes, Object Storage, and FCM Gateways

Summary: Architecture layout mapping production servers, static CDNs, and database nodes.

1. Production Infrastructure Deployment

Shows how client requests route to static and serverless nodes.



2. Operational Specifications

- **Vercel Global Edge CDN:** Serves static bundle assets, reducing latency.
- **Database Mirroring:** Database read replicas are configured to support heavy analytic aggregations.

CHAPTER 29

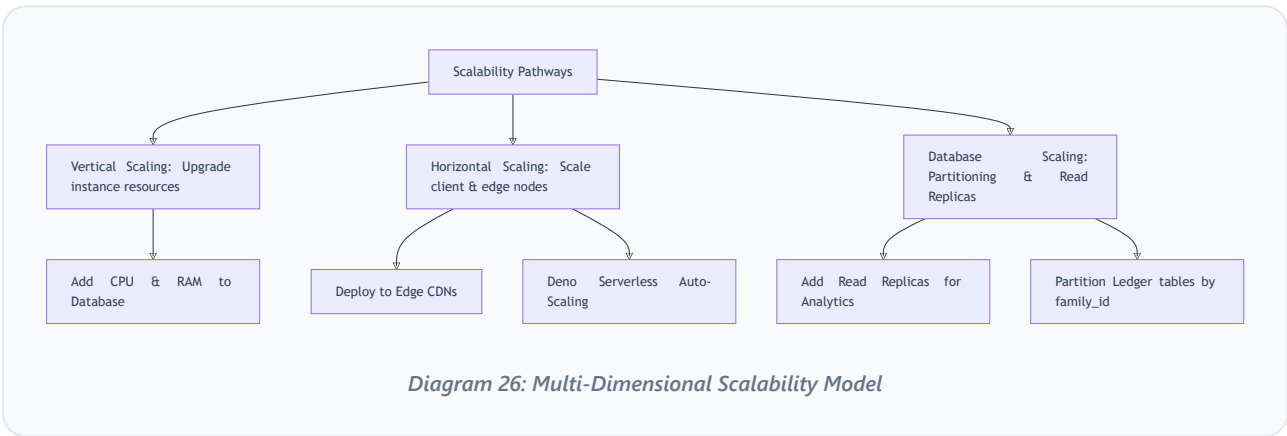
Horizontal & Vertical Scalability

Scale-up, Scale-out, Replication, and Partitioning Strategy

Summary: Outlines scaling strategies for the database, cache, and API gateways.

1. System Scaling Strategy

Outlines the vertical, horizontal, and database scaling strategies.



2. Scaling Implementation

Horizontal API Scaling

Supabase Edge Functions execute in isolated Deno isolates, scaling horizontally to meet demand.

Database Partitioning

Large tables like `transactions` can be partitioned logically by `family_id` to distribute the read/write load.

CHAPTER 30

Performance Tuning & Optimization

Code Splitting, Image Compression, and Query Indexes

Summary: Details the caching policies, index strategies, and code optimizations.

1. Code Splitting

Dynamic imports are used to compile views (such as `Analytics` and `Loans`) into separate chunks:

src/App.tsx

TYPESCRIPT

```
const Analytics = React.lazy(() => import('./pages/Analytics'));  
const Loans = React.lazy(() => import('./pages/Loans'));
```

2. Query Index Tuning

Indexes are placed on columns that are queried frequently:

supabase/migrations/002_user_preferences.sql

SQL

```
CREATE INDEX IF NOT EXISTS idx_transactions_member_date  
ON public.transactions(member_id, date DESC);
```

CHAPTER 31

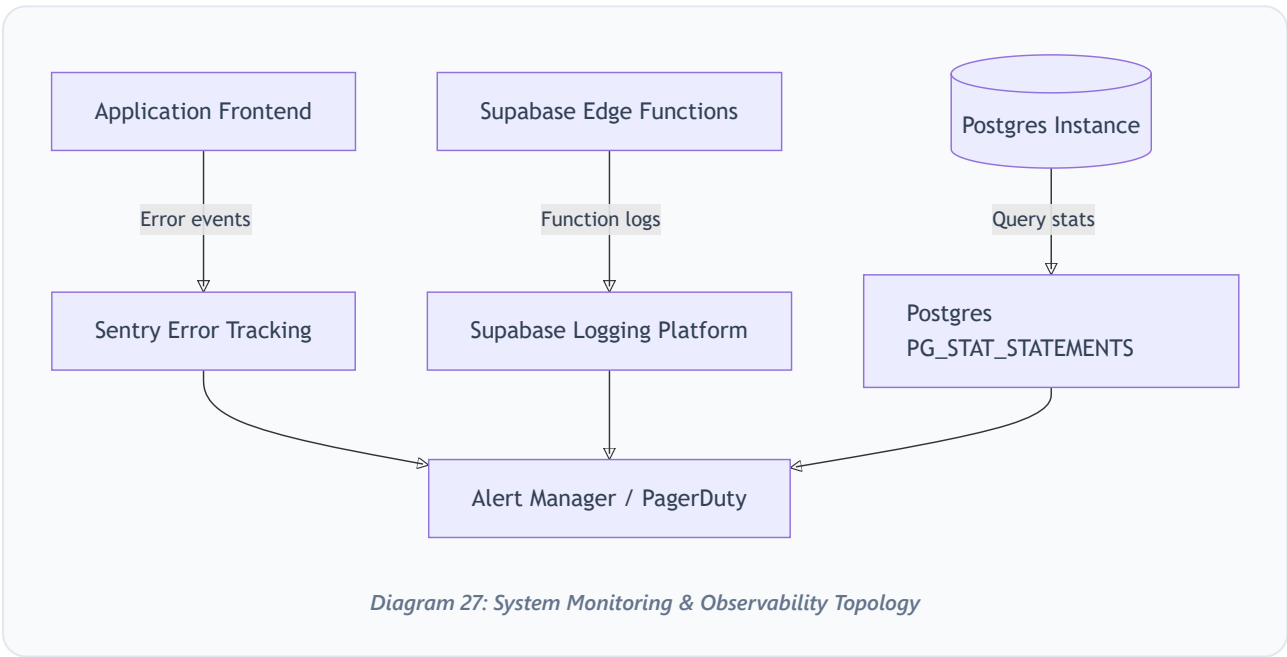
Observability & Monitoring

Error tracking, Edge function logs, and Query performance metrics

Summary: Details the logging and diagnostic systems used to monitor application health.

1. Monitoring Topology

Shows the flow of error logs from client to storage.



CHAPTER 32

Verification & Testing Strategy

Unit testing, Integration testing, and E2E browser flows

Summary: Outlines the testing frameworks and test coverage targets.

1. Testing Framework Structure

Shows the test types and associated frameworks.

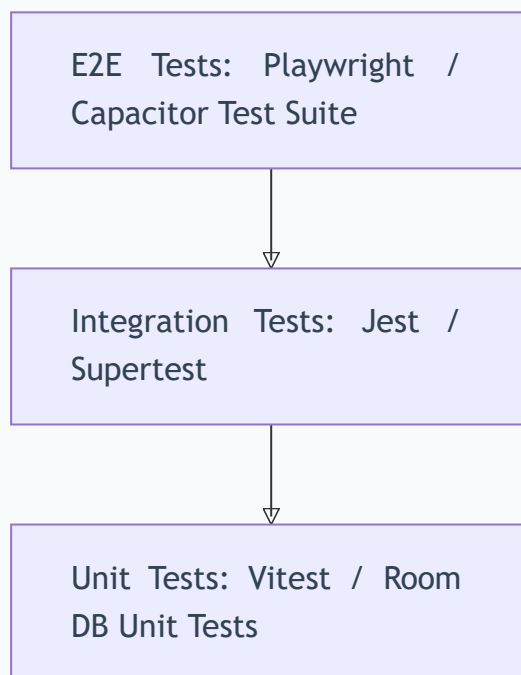


Diagram 28: Testing Lifecycle Pyramid

2. Test Execution

Running Frontend Tests

```
# Execute unit and component tests with Vitest  
npm run test:unit
```

Running Native Android Tests

```
# Execute unit tests in the Android project  
./gradlew testDebugUnitTest
```


CHAPTER 33

Codebase Directory Tree

Codebase Structure, Module Locations, and Build Configs

Summary: Maps the folder tree, codebase files, and configuration directories.

```
MS_Family_Workspace/
|
├── android/                                # Android Studio Native project files
|   └── app/src/main/java/com/msfamily/app/
|       ├── BackgroundLocationWorker.kt    # Geolocation sync worker
|       ├── BiometricAuthPlugin.java      # Native biometric app lock
|       ├── SmsBackgroundReceiver.java     # Intercepts incoming SMS
|       ├── SmsParserEngine.kt            # Regular expression parser
|       ├── SmsSyncWorker.kt              # Local-to-cloud SMS sync worker
|       ├── TransactionCachePlugin.kt      # SQLite database interface
|       └── TransactionDatabase.kt         # Room database initializer
|
├── supabase/                              # Supabase configuration & migrations
|   ├── functions/                         # Deno Edge Functions
|   │   ├── send-notification/
|   │   └── daily-rates-push/
|   └── migrations/                       # DB migrations and triggers
|
├── src/                                   # React Frontend Source files
|   ├── components/                       # Reusable UI elements
|   │   ├── ui/                          # shadcn UI components
|   │   └── Layout.tsx                   # Main navigation shell
|   ├── context/                         # React Context providers
|   │   ├── AuthContext.tsx             # User session manager
|   │   ├── FinanceContext.tsx          # Ledger state manager
|   │   ├── FamilyContext.tsx           # Group settings broker
|   │   └── CallContext.tsx             # WebRTC voice coordinator
|   ├── pages/                           # Page components
|   │   ├── Dashboard.tsx               # Primary home dashboard
|   │   ├── Transactions.tsx            # Ledger list interface
|   │   └── MyProofs.tsx                # Secure document vault
```

		—	Loans.tsx	# Loan & EMI tracking interface
		—	LiveTracking.tsx	# Geolocation sharing map
		—	utils/	# Helper libraries & services
—			package.json	# Dependencies configuration
—			tailwind.config.js	# CSS design configuration
—			vite.config.ts	# Vite build parameters

CHAPTER 34

Configuration & Environments

Environment Variables, Deployment Modes, and Key Security

Summary: Details the environment variables, settings, and key configurations.

Environment Variables Configuration

Table 7: Environment Variables Registry

VARIABLE NAME	EXAMPLE VALUE	DESCRIPTION	SECURITY RECOMMENDATION
VITE_SUPABASE_URL	https://x.supabase.co	API URL for the Supabase instance.	Publicly visible in client bundle.
VITE_SUPABASE_ANON_KEY	eyJhbGciOi...	Anonymous public API key.	Encrypted but visible in client bundle.
VITE_GEMINI_API_KEY	AIzaSy...	Developer API key for Google Gemini.	Should be routed through an Edge Function.
FCM_SERVER_KEY	AAAA...	Firebase Cloud Messaging server key.	Keep Secret. Store only in Edge Functions.
CLERK_PUBLISHABLE_KEY	pk_live...	Clerk authentication keys.	Publicly visible in client bundle.

CHAPTER 35

Design System & Tokens

Typography scales, Palette hexes, Spacing, and Button Tokens

Summary: Design specifications, custom colors, and typography layouts.

1. Color Palette

- **Primary (Deep Navy):** #0b1530
- **Secondary (Indigo):** #4f46e5
- **Accent (Emerald):** #10b981
- **Support (Slate Gray):** #64748b

2. Typography

- **Document Title:** 32pt Bold, Line Height 1.15
- **Heading 1:** 24pt Bold, Line Height 1.2
- **Heading 2:** 15pt Semi-Bold, Line Height 1.3
- **Body Text:** 10pt Regular, Line Height 1.625
- **Monospace / Code:** 8.5pt Fira Code, Line Height 1.5

CHAPTER 36

UML Core Blueprints

UML Use Case, Class, Sequence, and Deployment Diagrams

Summary: System architecture blueprints illustrating user actions, component layouts, and deployment topologies.

1. Use Case Diagram

Outlines how users interact with various modules.

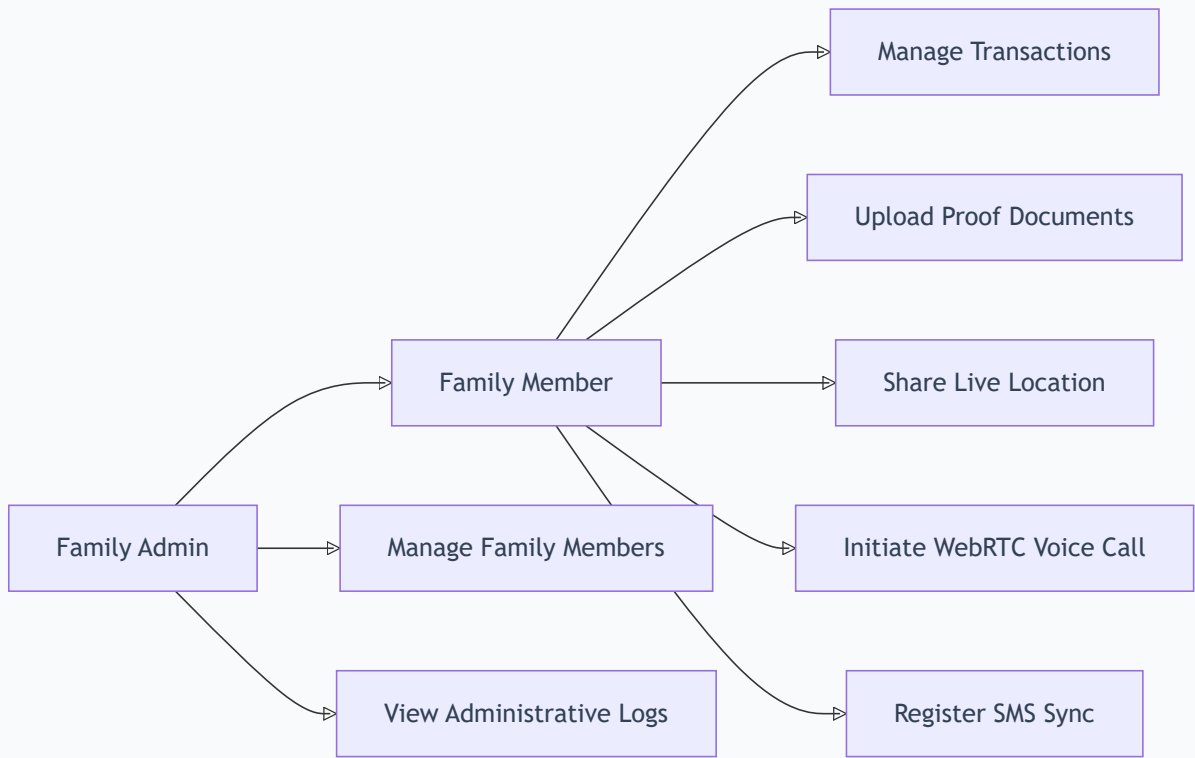
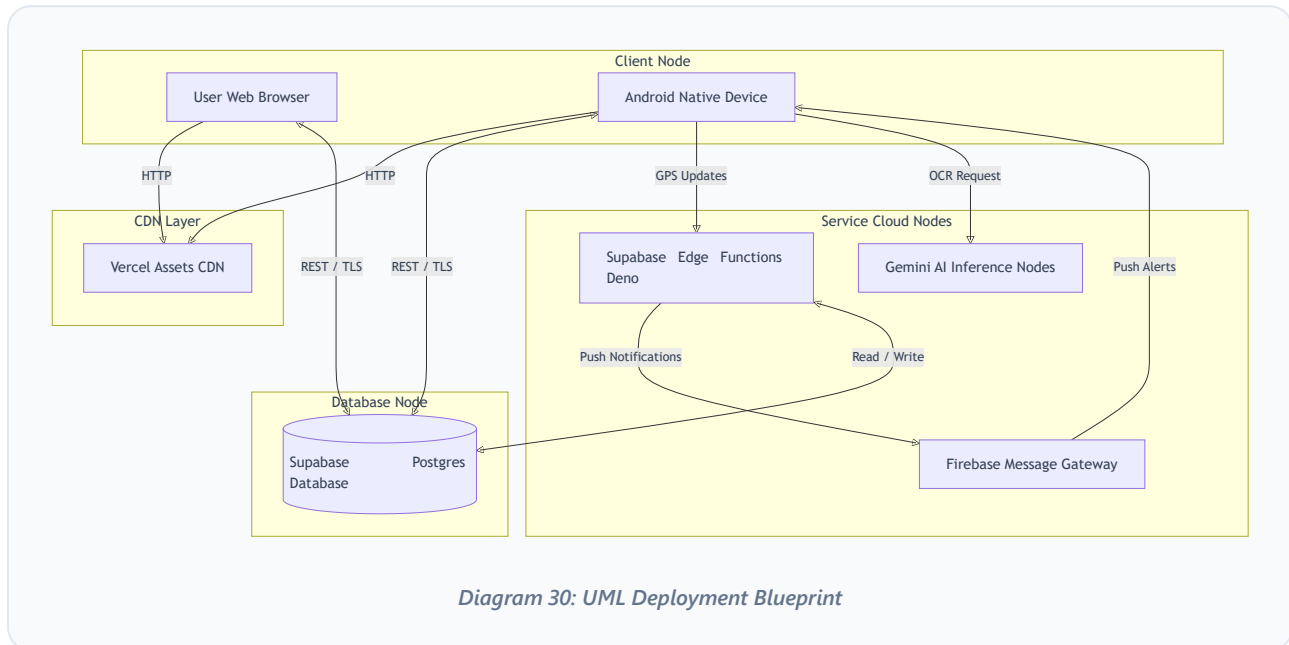


Diagram 29: UML Use Case Blueprint

2. System Deployment Diagram

Shows the physical distribution of system components.



CHAPTER 37

Architecture Decision Records (ADRs)

Technology selections, framework justifications, and architecture records

Summary: Details the ADR records for framework and database selections.

ADR-001: Selection of React + Vite for Frontend Development

Status

Approved

Context

We needed a fast, component-based frontend framework to support a responsive web dashboard and hybrid mobile application compiled via Capacitor.

Decision

React 18 was chosen for its component model and state management contexts. Vite was selected as the build tool to replace Webpack, providing faster development builds and optimized production assets.

Consequences

- **Positives:** Faster builds, clean module imports, and a component architecture that supports state management.
- **Negatives:** Relies on client-side rendering, which requires caching strategies to optimize initial load times.

ADR-002: Selection of Supabase (PostgreSQL) for Database & Auth

Status

Approved

Context

We needed a database that supports relational queries, user authentication, storage, and real-time data sync without requiring a custom API server.

Decision

Supabase was selected. It provides a managed PostgreSQL database with Row-Level Security (RLS), authentication, storage, and real-time subscription engines.

Consequences

- **Positives:** Built-in RLS policies simplify security, database changes sync automatically, and storage is integrated out of the box.
- **Negatives:** Increases dependency on a single service provider. Complex logic must be handled using PL/pgSQL database functions or Edge Functions.

CHAPTER 38

AI & LLM Engineering Pattern

Prompt configurations, JSON schema validations, and fallback parser structures

Summary: Details the prompt configurations, schemas, and fallback parser structures.

1. Gemini Ingestion Prompt Structure

When processing receipts in `MyProofs`, the system sends the image payload to Gemini with a structured prompt:

prompt_templates/receipt_extraction.txt

TEXT

Return ONLY a valid JSON object matching the schema below. Do not include markdown formatting

SCHEMA:

{

"amount": number,

"category": "Food" | "Utilities" | "Health" | "Travel" | "Other",

"merchantName": string,

"documentNumber": string or null,

"date": "YYYY-MM-DD" or null,

"summary": "1-sentence summary of the document contents"

}

2. Parsing & Recovery Flow

The frontend validates the JSON format of the response from Gemini. If the response contains markdown code fences, they are removed. If parsing fails, the system falls back to basic regex extraction before prompting the user for manual entry:

src/pages/MyProofs.tsx

JAVASCRIPT

```
// Example recovery parsing helper in MyProofs.tsx
function parseGeminiJson(text: string): any {
  let cleanText = text.trim();
  if (cleanText.startsWith('```')) {
    cleanText = cleanText.replace(/^```(?:json)?
?/, '').replace(/
?```$/ , '');
  }
  return JSON.parse(cleanText);
}
```

CHAPTER 39

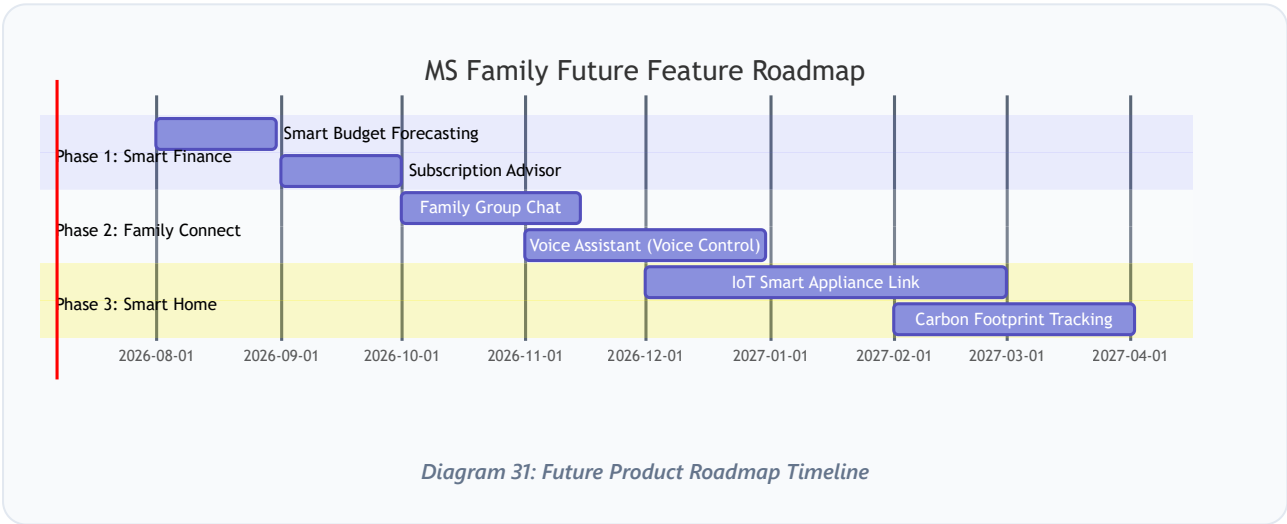
Product Evolution Roadmap

Phased development plans, forecasts, and integrations

Summary: Future feature plans, forecasting integrations, and IoT linkages.

1. Roadmap Timeline

Shows the roadmap for future releases.



2. Future Module Definitions

- **Smart Budget Forecasting:** Uses historical transaction data to predict future household expenses.
- **IoT Smart Appliance Link:** Integrates with smart utility meters to track home energy usage on the dashboard.

CHAPTER 40

Appendix & Bibliography

Glossary terms, version references, risks, and changes

Summary: Technical definitions, references, and project histories.

1. Glossary & Technical Definitions

- **Row-Level Security (RLS):** Database-layer security policy that isolates table rows based on user claims.
- **Capacitor:** Cross-platform hybrid runtime bridge linking JS SPAs with native mobile wrappers.
- **WebRTC:** Peer-to-peer real-time communication protocols for audio, video, and data channels.
- **FCM (Firebase Cloud Messaging):** Push notification routing service.

2. Bibliography & References

1. *PostgreSQL Row Level Security Documentation*, <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
2. *Capacitor Android Plugin Guide*, <https://capacitorjs.com/docs/plugins/android>
3. *WebRTC Architectures and Protocol Specifications*, RFC 7478.